

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 «Математика и компьютерные науки»

Отчёт по лабораторным работам №1–5

По дисциплине: «Теория графов»

На тему: «Алгоритмы на графах»

Вариант №44

Выполнил студент
группы 5130201/40002

_____ Семенов И. А.

Проверил
преподаватель

_____ Востров А. В.

«_____» _____ 2026г.

Содержание

Введение	3
1 Постановка задачи	4
2 Математическое описание	7
2.1 Генерация графа и основные понятия	7
2.2 Связность и точки сочленения	12
2.3 Максимальный поток и поток минимальной стоимости . .	16
2.4 Осто́вы, код Прюфера и независимые множества рёбер . .	19
2.5 Эйлеровы циклы и фундаментальная система разрезов . .	22
3 Особенности реализации	24
3.1 Лабораторная работа № 1	24
3.2 Лабораторная работа № 2	28
3.3 Лабораторная работа № 3	32
3.4 Лабораторная работа № 4	38
3.5 Лабораторная работа № 5	51
4 Результаты работы	58

Введение

Теория графов — раздел дискретной математики, изучающий объекты и связи между ними с помощью графов, состоящих из вершин и рёбер. В рамках теории графов рассматриваются задачи анализа связности, поиска путей, построения остовов, исследования циклов, потоков и паросочетаний. Графовые модели применяются при описании транспортных и компьютерных сетей, социальных связей, структур данных и других сложных систем.

В отчёте представлено описание выполнения цикла лабораторных работ по дисциплине «Теория графов». Целью работ является изучение способов представления графов и практическая реализация алгоритмов для их генерации, анализа и преобразования.

Разработанная консольная программа на языке C++ объединяет результаты пяти лабораторных работ. Программа выполняет случайную генерацию графа и весовых матриц, вычисляет эксцентриситеты вершин, выделяет центр графа и диаметральные вершины, ищет маршруты, кратчайшие пути, точки сочленения и потоки, строит минимальный остов, кодирует его кодом Прюфера, находит максимальное независимое множество рёбер, строит эйлеров цикл и фундаментальную систему разрезов. Исходный граф, сформированный в первой работе, затем используется в последующих работах.

В отчёте рассмотрены математическое описание использованных алгоритмов, особенности их программной реализации и результаты работы программы на сгенерированных графах.

1 Постановка задачи

Цель работы: разработать единый программный комплекс с текстовым пользовательским интерфейсом для генерации, анализа и преобразования графов, объединяющий алгоритмы теории графов, реализованные в ходе выполнения пяти лабораторных работ.

Задачи.

Первая лабораторная работа:

1. сформировать случайным образом связный ациклический ориентированный граф в соответствии со смещённым распределением Вейбулла с заданным пользователем количеством вершин;
2. реализовать генерацию весовой матрицы в соответствии со смещённым распределением Вейбулла с положительными, отрицательными или смешанными весами при заданном пользователем количестве рёбер;
3. посчитать эксцентриситеты вершин;
4. выделить центр графа;
5. определить диаметральные вершины;
6. реализовать метод Шимбелла для сгенерированной весовой матрицы при заданном пользователем количестве рёбер;
7. выводить максимальный или минимальный путь, полученный методом Шимбелла, в виде весовой матрицы;
8. определить возможность построения маршрута между двумя заданными пользователем вершинами и посчитать количество таких маршрутов;
9. реализовать пользовательское меню;

Вторая лабораторная работа:

10. найти точки сочленения в сгенерированном графе;
11. найти кратчайший путь между выбранными вершинами с помощью алгоритма Дейкстры для отрицательных весов, вывести расстояние, вектор расстояний и последовательность вершин пути;

12. сравнить скорость работы алгоритмов по количеству итераций;

Третья лабораторная работа:

13. сформировать случайным образом матрицу пропускных способностей в соответствии со смещённым распределением Вейбулла на основе сгенерированного ориентированного графа;
14. сформировать случайным образом матрицу стоимостей в соответствии со смещённым распределением Вейбулла на основе сгенерированного ориентированного графа;
15. найти максимальный поток для полученной матрицы пропускных способностей по алгоритму Форда–Фалкерсона;
16. вычислить поток минимальной стоимости величины $\lfloor 2/3 \cdot \max \rfloor$, здесь $\max$ — найденный максимальный поток;

Четвёртая лабораторная работа:

17. найти число остовных деревьев неориентированного графа с помощью матричной теоремы Кирхгофа;
18. построить минимальный по весу остов для неориентированного взвешенного графа алгоритмом Краскала;
19. закодировать остов кодом Прюфера с сохранением весов рёбер и декодировать его;
20. найти максимальное независимое множество рёбер на исходном графе и полученном остове по выбору пользователя;

Пятая лабораторная работа:

21. проверить, является ли неориентированный граф эйлеровым;
22. при необходимости модифицировать граф с сохранением связности и вывести сведения о выполненных изменениях;
23. построить эйлеров цикл;
24. построить минимальный остов алгоритмом Краскала;
25. получить фундаментальную систему разрезов и вывести её на экран;

26. получить остальные разрезы из фундаментальных с помощью операции симметрической разности для выбранных пользователем разрезов.

2 Математическое описание

2.1 Генерация графа и основные понятия

Графом $G(V, E)$ называется совокупность двух множеств:

- непустое множество V — множество вершин;
- E , множество двуэлементных подмножеств множества V , — множество рёбер.

Ребро можно трактовать не только как множество $\{v_1, v_2\}$, но и как пару вершин (v_1, v_2) .

$$p \stackrel{\text{Def}}{=} p(G) \stackrel{\text{Def}}{=} |V|, \quad q \stackrel{\text{Def}}{=} q(G) \stackrel{\text{Def}}{=} |E|.$$

Если рёбра задаются упорядоченными парами, то граф называется ориентированным. В противном случае — неориентированным. Степенью вершины $d(v)$ называется количество рёбер, инцидентных этой вершине.

Для неориентированного графа справедлива лемма о рукопожатиях. Сумма степеней вершин графа равна удвоенному количеству рёбер.

$$\sum_{v \in V} d(v) = 2q.$$

Маршрутом в графе называется чередующаяся последовательность вершин и рёбер, начинающаяся и заканчивающаяся вершиной, в которой любые два соседних элемента инцидентны.

- Если все рёбра различны, то маршрут называется цепью.
- Если все вершины различны, то маршрут называется простой цепью.

Если $v_0 = V_k$, то маршрут замкнут, иначе — открыт.

Цепь, соединяющая вершины u и v , обозначается $\langle u, v \rangle$. Говорят, что две вершины в графе связны, если существует соединяющая их цепь. *Граф, в котором все вершины связны, называется связным.*

Замкнутая цепь называется циклом, замкнутая простая цепь называется простым циклом. Число циклов в графе обозначается $z(G)$. *Граф без циклов называется ациклическим.*

В первой лабораторной работе связный ациклический граф генерируется случайным образом на основе смещённого распределения Вейбулла. Распределение задаётся функцией, приведённой в справочнике Р. Н. Вадзинского:

$$F(y) = 1 - \exp \left(- \left(\frac{y - y_0}{a} \right)^c \right),$$

где y — случайная величина, a — параметр масштаба, c — параметр формы ($c > 0$), y_0 — параметр смещения.

Для генерации случайных чисел по смещённому распределению Вейбулла используется формула

$$y = y_0 + a(-\ln r)^{1/c}.$$

Здесь r — равномерно распределённая на интервале $(0, 1)$ случайная величина.

На рис. 1 показана функция риска распределения Вейбулла при параметрах $a = 1$, $c = 0,5; 1,5; 2,5$, $y_0 = 0$.

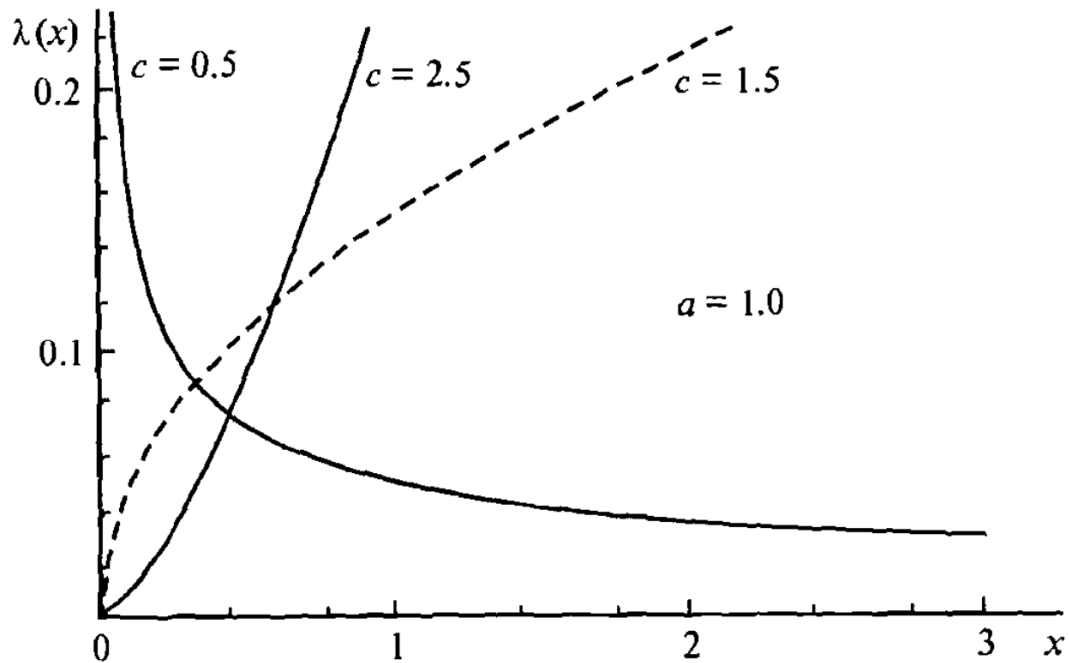


Рис. 1: Функция риска распределения Вейбулла

На рис. 2 приведены гистограммы 1000 сгенерированных значений для тех же параметров. При $c < 1$ большая часть значений сосредоточена возле нуля. При увеличении c значения становятся более сконцентрированными, а максимум гистограммы смещается вправо.

Смещённое распределение Вейбулла при различных значениях c

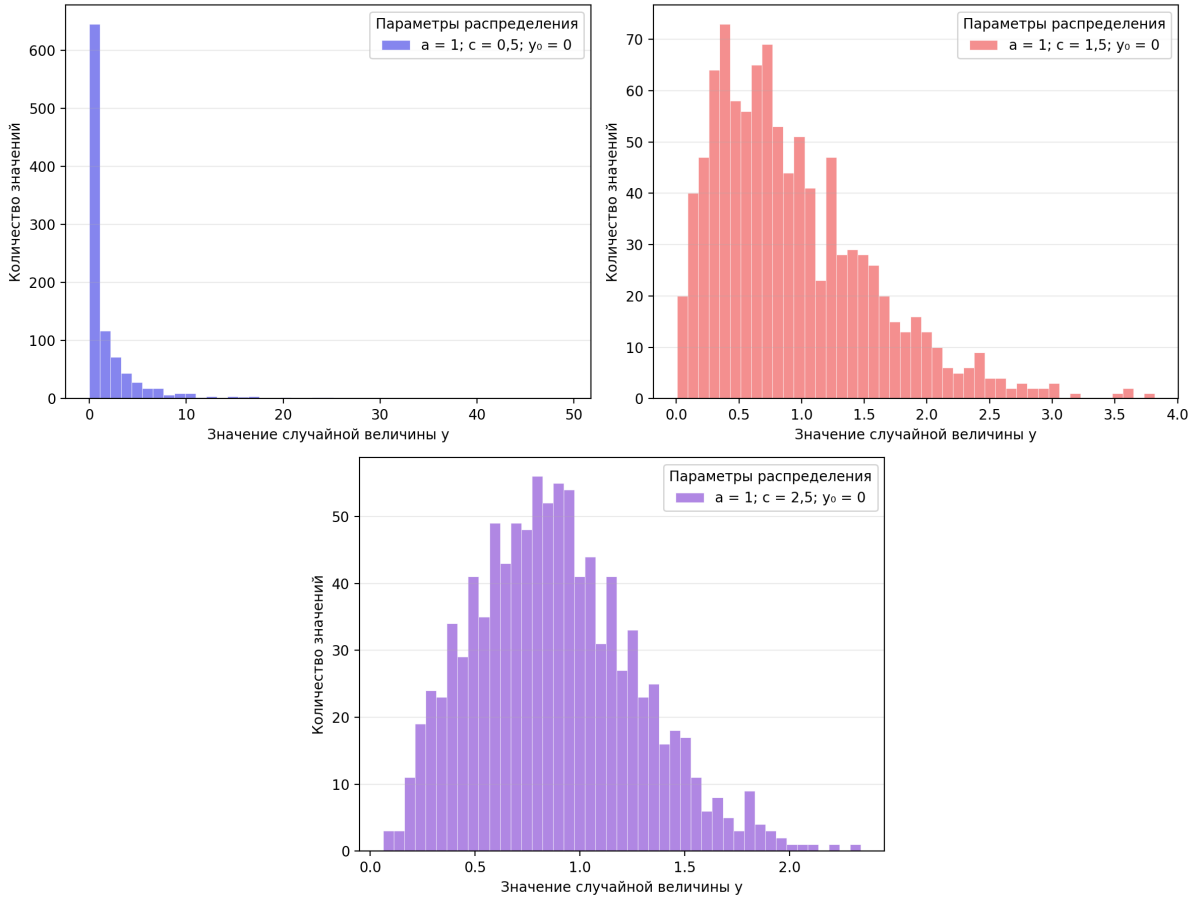


Рис. 2: Гистограммы смещённого распределения Вейбулла при различных значениях c

Для генерации матрицы смежности использовались параметры $a = 0.17$, $c = 2.0$, $y_0 = 0.05$. При формировании графа значение степени i -й вершины вычисляется по формуле

$$d_i = \text{round}((p - 1)y_i), \quad i = 1, \dots, p,$$

где p — количество вершин графа, y_i — значение случайной величины, полученное по смещённому распределению Вейбулла. Для графа из 10 вершин формула принимает вид

$$d_i = \text{round}(9y_i).$$

Граф строится по полученной последовательности степеней вершин при выполнении условия

$$1 \leq d_i \leq p - 1 \quad \text{для всех } i = 1, \dots, p.$$

Если хотя бы одна степень не удовлетворяет этому условию, последовательность генерируется заново. Такой набор параметров подобран

экспериментально. Он обеспечивает умеренные степени вершин и не приводит к формированию почти полного графа.

Гистограмма 1000 значений распределения с выбранными параметрами приведена на рис. 3. На том же рисунке показаны рассчитанные значения степеней для графа из 10 вершин.

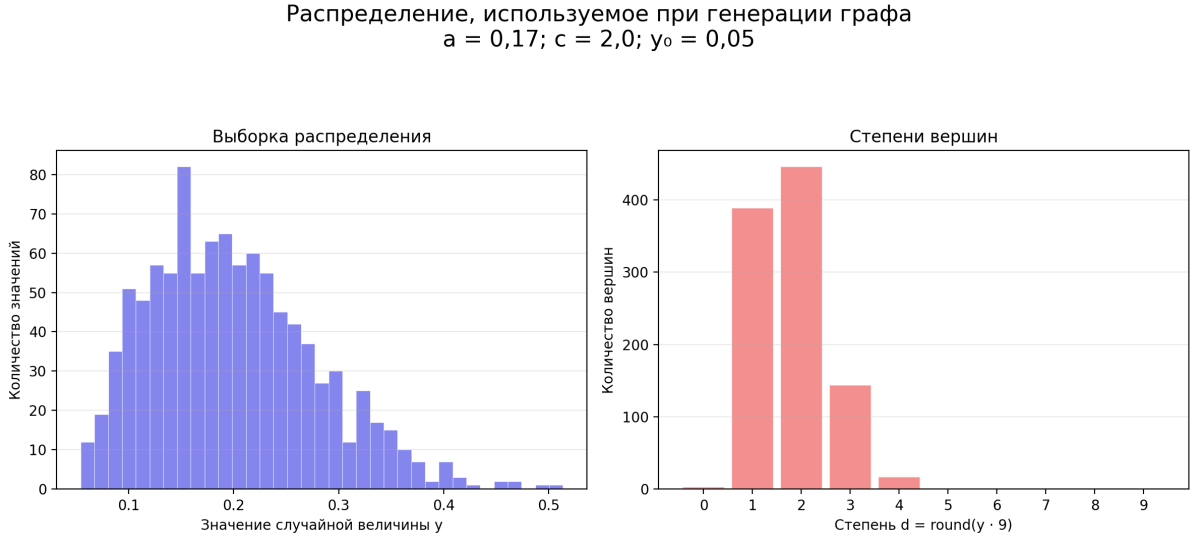


Рис. 3: Распределение Вейбулла и значения степеней при $p = 10$

В рамках работы смещённое распределение Вейбулла используется также для генерации весов рёбер и сети потоков. Для весовой матрицы пользователь может выбрать знак генерируемых весов: положительные, отрицательные или смешанные. При генерации со смешанными весами знак выбирается случайно.

Расстоянием между вершинами u и v (обозначается $d(u, v)$) называется длина кратчайшей цепи $\langle u, v \rangle$, а сама кратчайшая цепь называется геодезической.

$$d(u, v) \stackrel{\text{Def}}{=} \min_{\langle u, v \rangle} \langle u, v \rangle.$$

$$\text{Если } \nexists \langle u, v \rangle, \text{ то } d(u, v) \stackrel{\text{Def}}{=} +\infty.$$

Диаметром графа G называется длиннейшая геодезическая. Длина диаметра обозначается $D(G)$:

$$D(G) \stackrel{\text{Def}}{=} \max_{u, v \in V} d(u, v).$$

Эксцентриситетом $e(v)$ вершины v в связном графе $G(V, E)$ называется максимальное расстояние от v до любой другой вершины графа G :

$$e(v) \stackrel{\text{Def}}{=} \max_{u \in V} d(u, v).$$

Радиусом $R(G)$ графа G называется наименьший из эксцентриситетов вершин:

$$R(G) \stackrel{\text{Def}}{=} \min_{v \in V} e(v).$$

Вершина v называется центральной, если её эксцентриситет совпадает с радиусом графа, т.е. $e(v) = R(G)$.

Множество центральных вершин называется центром графа и обозначается $C(G)$:

$$C(G) \stackrel{\text{Def}}{=} \{v \in V \mid e(v) = R(G)\}.$$

Метод Шимбелла использует степени весовой матрицы для нахождения весов маршрутов с фиксированным количеством рёбер. В данной работе для каждой пары вершин определяется минимальный или максимальный суммарный вес маршрута, содержащего ровно k рёбер.

Пусть $W = (w_{ij})$ — весовая матрица графа. При Шимбелловом умножении произведение элементов заменяется сложением весов, а сложение полученных значений — выбором минимума или максимума. Правила выполнения операций приведены на рис. 4.

1. Операция умножения двух величин a и b при возведении матрицы в степень соответствует их алгебраической сумме, то есть

$$\begin{cases} a \cdot b = b \cdot a \Rightarrow a + b = b + a, \\ a \cdot 0 = 0 \cdot a = 0 \Rightarrow a + 0 = 0. \end{cases} \quad (3.8.1)$$

2. Операция сложения двух величин a и b заменяется выбором из этих величин минимального (максимального) элемента, то есть

$$a + b = b + a = \min(\max) \{a, b\} \quad (3.8.2)$$

нули при этом игнорируются. Минимальный или максимальный элемент выбирается из ненулевых элементов. Нуль в результате операции (3.8.2) может быть получен лишь тогда, когда все элементы из выбираемых — нулевые.

$$\omega_{ij}^{(2)} = \min(\max) \{(\omega_{i1}^{(1)} + \omega_{1j}^{(1)}), (\omega_{i2}^{(1)} + \omega_{2j}^{(1)}), \dots, (\omega_{in}^{(1)} + \omega_{nj}^{(1)})\}$$

Рис. 4: Матричные операции метода Шимбелла

Для произвольного количества рёбер вычисления продолжаютс я рекуррентно. Произведение матриц в следующей формуле является Шимбелловым умножением и выполняется по правилам, приведённым на рисунке 4:

$$W^{(1)} = W, \quad W^{(k)} = W^{(k-1)} \cdot W.$$

Следовательно, элемент $w_{ij}^{(k)}$ результирующей матрицы содержит минимальный или максимальный вес маршрута из v_i в v_j , состоящего ровно из k рёбер. При $k = 0$ диагональные элементы матрицы равны нулю, а остальные элементы соответствуют отсутствующим маршрутам.

2.2 Связность и точки сочленения

Отношение связности вершин называется эквивалентностью. *Классы эквивалентности по отношению связности называются компонентами связности графа.*

Вершина графа называется точкой сочленения, если её удаление увеличивает число компонент связности. Мостом называется ребро, удаление которого увеличивает число компонент связности.

Для поиска точек сочленения граф обходится в глубину. Для каждой вершины определяется, существует ли из её поддерева путь к ранее посещённым вершинам в обход неё. Если такого пути нет, удаление этой вершины разделит граф, поэтому она является точкой сочленения. Для корневой вершины отдельно проверяется количество независимых ветвей обхода. В реализации ориентированный граф рассматривается как неориентированный.

Псевдокод обхода в глубину с вычислением нижних меток приведён на рис. 5.

```

1.  procedure BiComp(v);
2.  begin
3.    num[v] := i; L[v] := i; i := i + 1;
4.    for u ∈ list[v] do
5.      if num[u] = 0 then
6.        begin
7.          SE ← vu; father[u] := v; BiComp(u)
8.          L[v] := min(L[v], L[u]);
9.          if L[u] ≥ num[v] then
10.           {получить новый блок; для этого из
               стека SE надо вытолкнуть все ребра
11.         end
12.       else if num[u] < num[v] and u ≠ father[v] then
13.         begin
14.           SE ← vu; L[v] := min(L[v], num[u]);
15.         end
16.     end;
17.   begin
18.     i := 1; SE := nil; father[v0] := ∅;
19.     for v ∈ V do num[v] := 0;
20.     BiComp(v0)
21.   end.

```

Рис. 5: Псевдокод поиска точек сочленения

Стандартный алгоритм Дейкстры находит кратчайшие пути от начальной вершины s до остальных вершин графа с неотрицательными весами. На каждой итерации выбирается вершина с минимальным текущим весом пути. Затем веса путей до соседних вершин пересчитываются в соответствии с неравенством треугольника. После этого выбранная вершина считается обработанной окончательно. Псевдокод стандартного алгоритма приведён на рис. 6.

Вход: орграф $G(V, E)$, заданный матрицей длин дуг $C : \text{array } [1..p, 1..p] \text{ of real}$;
 s и t — вершины графа.

Выход: векторы $T : \text{array } [1..p] \text{ of real}$; и $H : \text{array } [1..p] \text{ of } 0..p$. Если вершина v лежит на кратчайшем пути от s к t , то $T[v]$ — длина кратчайшего пути от s к v ; $H[v]$ — вершина, непосредственно предшествующая v на кратчайшем пути.

```

for  $v$  from 1 to  $p$  do
   $T[v] := \infty$  { кратчайший путь неизвестен }
   $X[v] := 0$  { все вершины не отмечены }
end for
 $H[s] := 0$  {  $s$  ничего не предшествует }
 $T[s] := 0$  { кратчайший путь имеет длину 0... }
 $X[s] := 1$  { ... и он известен }
 $v := s$  { текущая вершина }
 $M := \{\}$  { обновление пометок }
for  $u \in \Gamma(v)$  do
  if  $X[u] = 0$  &  $T[u] > T[v] + C[v, u]$  then
     $T[u] := T[v] + C[v, u]$  { найден более короткий путь из  $s$  в  $u$  через  $v$  }
     $H[u] := v$  { запоминаем его }
  end if
end for
 $m := \infty$ ;  $v := 0$ 
{ поиск конца кратчайшего пути }
for  $u$  from 1 to  $p$  do
  if  $X[u] = 0$  &  $T[u] < m$  then
     $v := u$ ;  $m := T[u]$  { вершина  $v$  заканчивает кратчайший путь из  $s$  }
  end if
end for
if  $v = 0$  then
  stop { пет пути из  $s$  в  $t$  }
end if
if  $v = t$  then
  stop { найден кратчайший путь из  $s$  в  $t$  }
end if
 $X[v] := 1$  { найден кратчайший путь из  $s$  в  $v$  }
goto  $M$ 

```

Рис. 6: Псевдокод стандартного алгоритма Дейкстры

Для работы с отрицательными весами алгоритм модифицирован: вершина не фиксируется окончательно и при нахождении пути меньшего веса повторно помещается в очередь с приоритетами. Поэтому её вес может уменьшаться несколько раз. Сравнение стандартного и модифицированного алгоритмов приведено на рис. 7.

- Для графов с отрицательными весами (без циклов с суммарным отрицательным весом) время выполнения алгоритма экспоненциальное.

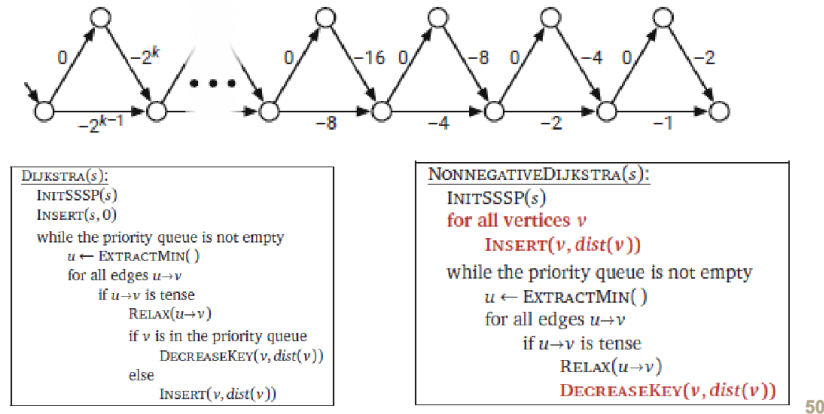


Рис. 7: Сравнение алгоритмов для неотрицательных и отрицательных весов

Для защиты от закливания алгоритм подсчитывает количество уменьшений веса каждой вершины. Если для некоторой вершины оно достигает количества вершин графа p , выполнение прекращается и фиксируется наличие отрицательного цикла. При его отсутствии алгоритм завершается после опустошения очереди. Пример изменения весов приведён на рис. 8 и 9.

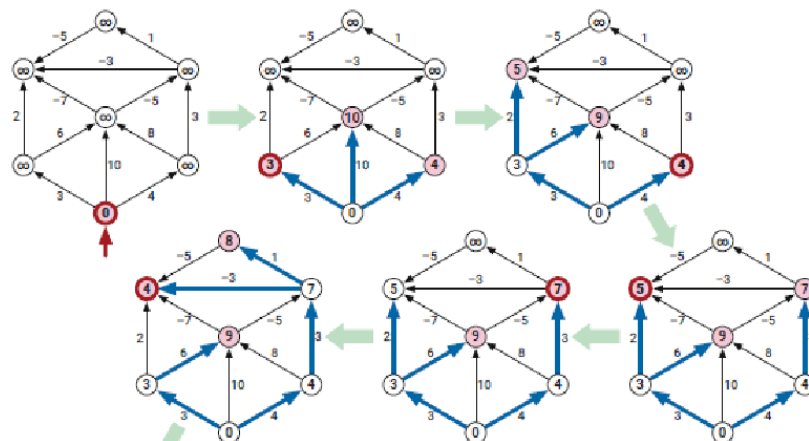


Рис. 8: Последовательное изменение весов при работе алгоритма, часть 1

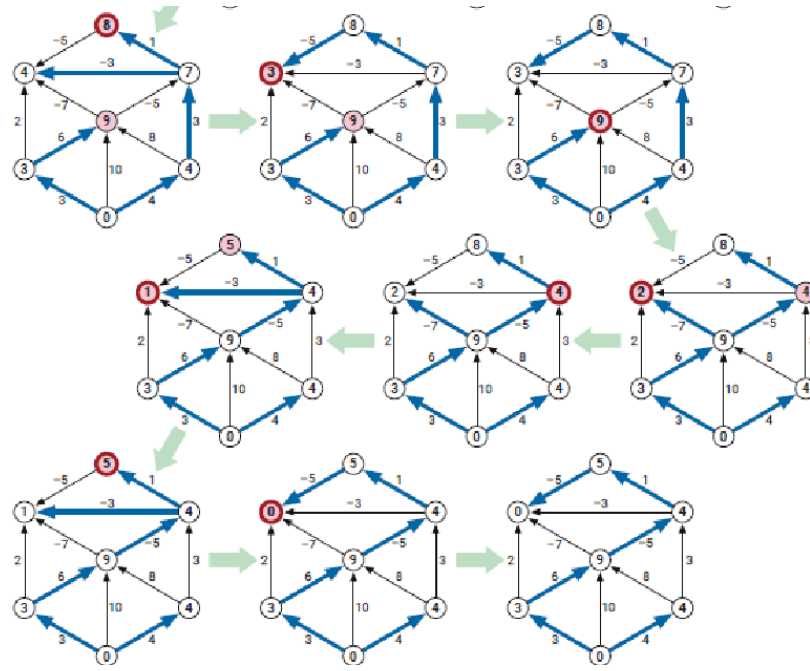


Рис. 9: Последовательное изменение весов при работе алгоритма, часть 2

Результаты сравнения алгоритмов по количеству итераций и соответствующие графики приведены в [разделе «Результаты работы»](#). По результатам запусков поиск точек сочленения оказался эффективнее модифицированного алгоритма Дейкстры.

2.3 Максимальный поток и поток минимальной стоимости

Пусть $G = (V, E)$ — орграф с одним истоком $s \in V$ и одним стоком $t \in V$, где $s \neq t$.

Сетью $S = (G; c)$ называется орграф G с заданной функцией $c : E \rightarrow \mathbb{R}_+$, сопоставляющей каждой дуге $e \in E$ неотрицательное число $c(e)$, называемое пропускной способностью этой дуги e .

Потоком в сети $S = (G; c)$, где $G = (V, E)$, называется такая функция $f : E \rightarrow \mathbb{R}_+$, приписывающая каждой дуге $e \in E$ неотрицательное число $f(e)$, называемое потоком по этой дуге e .

Для поиска максимального потока применяется алгоритм Форда — Фалкерсона. В данной работе он реализован с помощью расстановки меток. Сначала поток по всем дугам полагается равным нулю, а исток получает начальную метку. Далее метки распространяются по дугам, по которым поток можно увеличить, и по обратным дугам, по которым ранее проведённый поток можно уменьшить. Каждая метка хранит предыдущую вершину, направление изменения потока и наименьшую пропускную способность найденной цепи.

Если метка достигает стока, по сохранённым вершинам восстанавливается увеличивающая цепь и поток вдоль неё изменяется на найденную

величину. После этого метки расставляются заново. Когда сток больше нельзя пометить, увеличивающих цепей не остаётся и полученный поток является максимальным. Псевдокод алгоритма приведён на рис. 10 и 11.

Вход: сеть $G(V, E)$ с источником s и стоком t , заданная матрица пропускных способностей $C : \text{array } [1..p, 1..p] \text{ of real}$.
Выход: матрица максимального потока $F : \text{array } [1..p, 1..p] \text{ of real}$.
for $u, v \in V$ **do**
 $F[u, v] := 0$ { вначале поток нулевой }
end for
 $M : \{ \text{итерация увеличения потока} \}$
for $v \in V$ **do**
 $S[v] := 0; N[v] := 0; P[v] := (+, \text{nil}, 0)$ { инициализация }
end for
 $S[s] := 1; P[s] := (+, \text{nil}, \infty)$ { так как $s \in S$ }
repeat
 $a := 0$ { признак расширения S }
 for $v \in V$ **do**
 if $S[v] = 1 \ \& \ N[v] = 0$ **then**
 for $u \in \Gamma(v)$ **do**
 if $S[u] = 0 \ \& \ F[v, u] < C[v, u]$ **then**
 $S[u] := 1; P[u] := (+, v, \min(P[v].\delta, C[v, u] - F[v, u])); a := 1$
 end if
 end for
 end if
 end for
 $N[v] := 1$ { узел v отмечен }
end if
end for
if $S[t]$ **then**
 $x := t$ { текущий узел аугментальной цепи }
 $\delta := P[t].\delta$ { величина изменения потока }
 while $x \neq s$ **do**
 if $P[x].s = +$ **then**
 $F[P[x].n, x] := F[P[x].n, x] + \delta$ { увеличение потока }
 else
 $F[x, P[x].n] := F[x, P[x].n] - \delta$ { увеличение потока }
 end if
 $x := P[x].n$ { предыдущий узел аугментальной цепи }
 end while
 goto M
end if
until $a = 0$

Рис. 10: Псевдокод алгоритма Форда — Фалкерсона, часть 1

end for
for $u \in \Gamma^{-1}(v)$ **do**
 if $S[u] = 0 \ \& \ F[u, v] > 0$ **then**
 $S[u] := 1; P[u] := (-, v, \min(P[v].\delta, F[u, v])); a := 1$
 end if
end for
 $N[v] := 1$ { узел v отмечен }
end if
end for
if $S[t]$ **then**
 $x := t$ { текущий узел аугментальной цепи }
 $\delta := P[t].\delta$ { величина изменения потока }
 while $x \neq s$ **do**
 if $P[x].s = +$ **then**
 $F[P[x].n, x] := F[P[x].n, x] + \delta$ { увеличение потока }
 else
 $F[x, P[x].n] := F[x, P[x].n] - \delta$ { увеличение потока }
 end if
 $x := P[x].n$ { предыдущий узел аугментальной цепи }
 end while
 goto M
end if
until $a = 0$

Рис. 11: Псевдокод алгоритма Форда — Фалкерсона, часть 2

Последовательное увеличение потока на примере сети показано на рис. 12.

Рассмотрим сеть S , и пусть f_0 — нулевой поток.

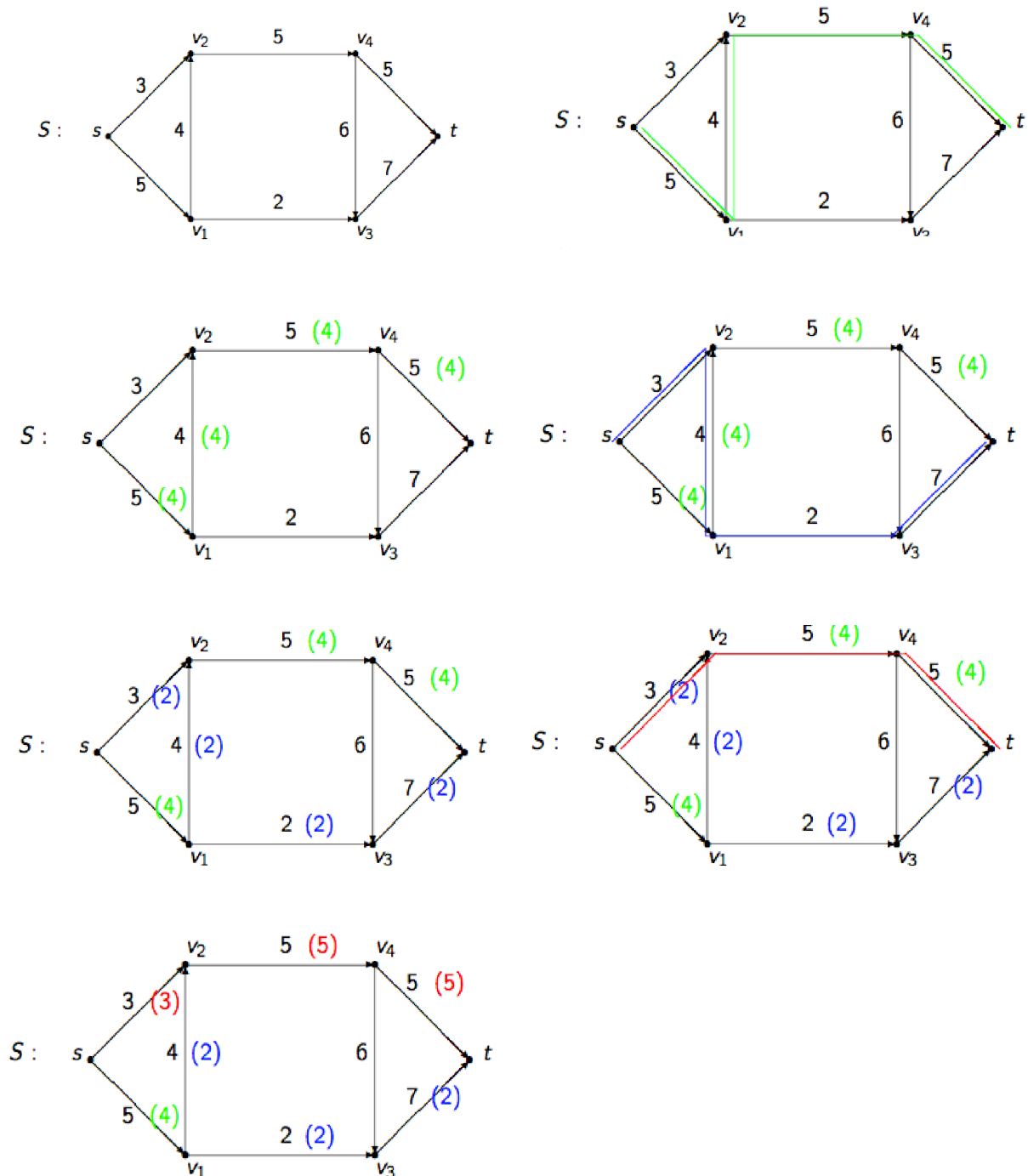


Рис. 12: Последовательное построение максимального потока

Пусть каждой дуге $e \in E$ также поставлена в соответствие стоимость $a(e)$ единицы проходящего по ней потока. Тогда стоимость потока f определяется суммой

$$C(f) = \sum_{e \in E} a(e)f(e).$$

В работе требуется найти поток минимальной стоимости заданной величины

$$F_{\text{req}} = \left\lfloor \frac{2}{3} F_{\text{max}} \right\rfloor,$$

где F_{max} — величина максимального потока, найденная алгоритмом Форда — Фалкерсона. Если $F_{\text{max}} = 1$, принимается $F_{\text{req}} = 1$, чтобы заданная величина потока оставалась положительной.

Поток минимальной стоимости строится в полученной ранее сети. На каждой итерации модифицированным алгоритмом Дейкстры находится путь наименьшей стоимости от истока к стоку. После этого величина потока увеличивается на максимально допустимое значение, определяемое пропускными способностями рёбер найденного пути. Вычисления продолжаются до достижения F_{req} либо до отсутствия такого пути.

2.4 Остовы, код Прюфера и независимые множества рёбер

Граф без циклов называется ациклическим, или лесом. Связный ациклический граф называется свободным деревом.

Пусть $G = (V, E)$ — граф. Остовным подграфом графа G называется подграф, содержащий все вершины графа G . Остовный подграф, являющийся деревом, называется остовом. Поскольку вершины остова совпадают с вершинами исходного графа, остов однозначно задаётся множеством своих рёбер. Несвязный граф не имеет остова, а связный граф может иметь несколько остовов.

Матрицей Кирхгофа графа G называется матрица $B = (b_{ij})$, диагональные элементы которой равны степеням соответствующих вершин, а остальные элементы равны -1 для смежных вершин и нулю в противном случае. По теореме Кирхгофа число остовов связного графа равно алгебраическому дополнению любого элемента матрицы B . Пример вычисления приведён на рис. 13.

- **Теорема Кирхгофа.** Число остовных деревьев в связном графе G порядка $n \geq 2$ равно алгебраическому дополнению любого элемента матрицы Кирхгофа $B(G)$.
- **Н.В.** Все алгебраические дополнения матрицы Кирхгофа равны между собой.

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix} \quad B = \begin{bmatrix} 2 & -1 & 0 & -1 & 0 & 0 \\ -1 & 5 & -1 & -1 & -1 & -1 \\ 0 & -1 & 2 & 0 & -1 & 0 \\ -1 & -1 & 0 & 3 & -1 & 0 \\ 0 & -1 & -1 & -1 & 3 & 0 \\ 0 & -1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_{23} = (-1)^{2+3} \begin{vmatrix} 2 & -1 & -1 & 0 & 0 \\ 0 & -1 & 0 & -1 & 0 \\ -1 & -1 & 3 & -1 & 0 \\ 0 & -1 & -1 & 3 & 0 \\ 0 & -1 & 0 & 0 & 1 \end{vmatrix} = (-1)^5 \begin{vmatrix} 2 & -1 & -1 & 0 & 0 \\ 0 & -1 & 0 & -1 & 0 \\ -1 & -1 & 3 & -1 & 0 \\ 0 & -1 & -1 & 3 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{vmatrix} = (-1)^{5+5+5} \begin{vmatrix} 2 & -1 & -1 & 0 \\ 0 & -1 & 0 & -1 \\ -1 & -1 & 3 & -1 \\ 0 & -1 & -1 & 3 \end{vmatrix} =$$

$$= (-1)^{15} \begin{vmatrix} 2 & -1 & -1 & 0 \\ 0 & 0 & 0 & -1 \\ -1 & 0 & 3 & -1 \\ 0 & -4 & -1 & 3 \end{vmatrix} = (-1)^{16+4+2} \begin{vmatrix} 2 & -1 & -1 \\ -1 & 0 & 3 \\ 0 & -4 & -1 \end{vmatrix} = (-1)^{22} \begin{vmatrix} 2 & -1 & 5 \\ -1 & 0 & 0 \\ 0 & -4 & -1 \end{vmatrix} = (-1)^{26} \begin{vmatrix} -1 & 5 \\ -4 & -1 \end{vmatrix} = 1 + 20 = 21$$

Рис. 13: Вычисление числа остовов по теореме Кирхгофа

Минимальным остовом взвешенного графа называется остов с наименьшей суммой весов рёбер. Алгоритм Краскала рассматривает рёбра в порядке возрастания их весов. В начале каждая вершина образует отдельное множество. Очередное ребро добавляется в остов, если его концы принадлежат разным множествам, после чего эти множества объединяются. Такая проверка не допускает образования циклов. После выбора $|V| - 1$ ребра получается минимальный остов. Псевдокод алгоритма и пример последовательного построения остова приведены на рис. 14.

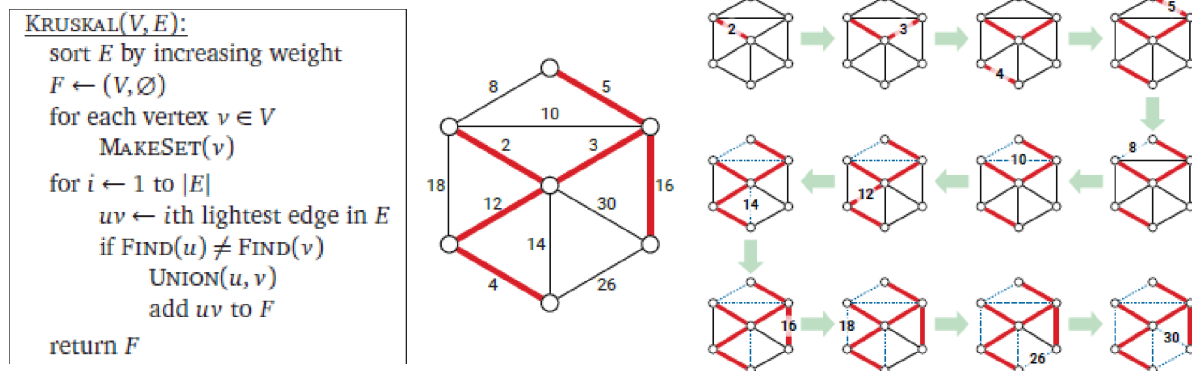


Рис. 14: Алгоритм Краскала и пример его работы

Множество рёбер называется независимым, или паросочетанием, если никакие два его ребра не смежны. Наибольшее число рёбер в независимом множестве рёбер называется рёберным числом независимости и обозначается β_1 .

Вершина, не инцидентная ни одному ребру паросочетания, называется свободной. Увеличивающий путь начинается и заканчивается в свободных вершинах, а его рёбра попеременно не принадлежат и принадлежат паросочетанию. Замена входящих в паросочетание рёбер такого пути на не входящие увеличивает число выбранных рёбер на один.

Алгоритм Эдмондса ищет увеличивающие пути обходом в ширину. Если в дереве поиска обнаруживается цикл нечётной длины, он временно сжимается в одну вершину, после чего поиск продолжается. При нахождении увеличивающего пути сжатые циклы раскрываются, а паросочетание увеличивается. Если увеличивающих путей больше нет, полученное паросочетание является максимальным. Псевдокод алгоритма приведён на рис. 15.

Подробное описание и интерактивная визуализация алгоритма приведены в материале Технического университета Мюнхена “[Edmonds’s Blossom Algorithm](#)”.

Algorithm status

prev
next
fast forward

Explanation
Pseudocode

```

BEGIN
  WHILE F != ∅ DO      (*set of free nodes F*)
    pick r ∈ F
    queue.push(r)      (*BFS queue*)
    T ← ∅               (*BFS tree T*)
    T.add(r)
    WHILE queue != ∅
      v ← queue.pop()
      FOR ALL neighbors w of v DO
        IF w ∉ T AND w matched THEN
          T.add(w)
          T.add(mate(w))
          queue.push(mate(w))
        ELSE IF w ∈ T AND even-length
          cycle detected THEN
          CONTINUE
        ELSE IF w ∈ T AND odd-length
          cycle detected THEN
          contract cycle
        ELSE IF w ∈ F THEN
          expand all contracted nodes
          reconstruct augmenting path
          invert augmenting path
      END
    END
  END

```

Рис. 15: Псевдокод алгоритма Эдмондса

2.5 Эйлеровы циклы и фундаментальная система разрезов

Если граф имеет цикл (не обязательно простой), содержащий все рёбра графа, то такой цикл называется эйлеровым циклом, а граф — эйлеровым графом. *Эйлеровым может быть только связный граф.*

Для приведения графа к эйлерову виду сначала находятся вершины нечётной степени. Из них выбираются две вершины u и v . Если они не смежны, ребро (u, v) добавляется. Если ребро существует, оно удаляется только при наличии обходного пути между u и v , что сохраняет связность графа. После каждой операции чётность степеней u и v меняется, поэтому число вершин нечётной степени уменьшается на два. Поскольку таких вершин всегда чётное число, процесс завершается за конечное число шагов.

Для построения эйлерова цикла применяется алгоритм Флёрри. Начальная вершина помещается в стек. Из текущей вершины выбирается соседняя вершина, после чего соединяющее их ребро удаляется, а выбранная вершина добавляется в стек. Переходы продолжаются, пока у текущей вершины не останется соседей. Такая вершина добавляется в результат и удаляется из стека, после чего обход продолжается из предыдущей вершины. После опустошения стека получается эйлеров цикл. Псевдокод алгоритма приведён на рис. 16.

Вход: эйлеров граф $G(V, E)$, заданный списками смежности ($\Gamma[v]$ — список вершин, смежных с вершиной v).

Выход: последовательность вершин эйлерова цикла.

```
 $S := \emptyset$  { стек для хранения вершин }  
select  $v \in V$  { произвольная вершина }  
 $v \rightarrow S$  { положить  $v$  в стек  $S$  }  
while  $S \neq \emptyset$  do  
   $v := \text{top } S$  {  $v$  — верхний элемент стека }  
  if  $\Gamma[v] = \emptyset$  then  
     $v \leftarrow S$ ; yield  $v$  { очередная вершина эйлерова цикла }  
  else  
    select  $u \in \Gamma[v]$  { взять первую вершину из списка смежности }  
     $u \rightarrow S$  { положить  $u$  в стек }  
     $\Gamma[v] := \Gamma[v] - u$ ;  $\Gamma[u] := \Gamma[u] - v$  { удалить ребро  $(v, u)$  }  
  end if  
end while
```

Если реализовать поиск ребер инцидентных вершине и удаление ребер за $O(1)$, то алгоритм будет работать за $O(p)$.

Рис. 16: Псевдокод алгоритма Флёрри

Пусть $T(V, E_T)$ — некоторый остов графа $G(V, E)$. Рассмотрим ребро остова e и определим разрез S_e следующим образом.

Ребро e — мост в дереве T . Следовательно, удаление ребра e разбивает множество вершин V на два непустых подмножества V_1 и V_2 , так что

$$V_1 \in V, V_1 \neq \emptyset, V_2 \in V, V_2 \neq \emptyset, V_1 \cup V_2 = V, V_1 \cap V_2 = \emptyset.$$

Включим в разрезе S_e ребро e и все хорды остова T , соединяющие вершины множества V_1 с вершинами множества V_2 , тогда S_e — простой разрез:

$$S_e \stackrel{\text{Def}}{=} e + \{(v_1, v_2) \in T \mid v_1 \in V_1 \ \& \ v_2 \in V_2\} = E(V_1, V_2).$$

Система разрезов называется фундаментальной системой разрезов. Разрезы фундаментальной системы называются фундаментальными, а количество разрезов в фундаментальной системе называется коциклическим рангом графа G и обозначается $m^*(G)$.

Рассмотрим векторное пространство над двоичной арифметикой, натянутое на множество рёбер E графа $G(V, E)$. Элемент этого векторного пространства (линейную комбинацию) можно отождествлять с подмножеством рёбер: если коэффициент в линейной комбинации равен 1, то ребро входит в подмножество, если коэффициент равен 0, то не входит.

Сложению векторов соответствует симметрическая разность множеств рёбер. Множество правильных разрезов замкнуто относительно симметрической разности, поэтому оно является подпространством общего пространства рёбер графа.

Множество разрезов называется независимым, если ни один из разрезов не является линейной комбинацией остальных.

Максимальное независимое множество разрезов является базисом подпространства разрезов. Фундаментальная система разрезов является базисом подпространства разрезов. Разрезы любой фундаментальной системы независимы.

Для построения фундаментальной системы сначала алгоритмом Краскала строится остов T . Затем из остова исключается ребро e , в результате чего остов разделяется на две части. В разрез S_e включаются исключённое ребро и все хорды, соединяющие вершины разных частей. Эти действия повторяются для каждого ребра остова, то есть $p - 1$ раз. Полученные разрезы образуют фундаментальную систему.

3 Особенности реализации

Основные алгоритмы реализованы методами класса `GeneratorGraph`. Взаимодействие с пользователем организовано классом `CLI` через общее консольное меню.

Далее через p обозначается количество вершин графа, через q — количество его рёбер, а через k — заданное количество рёбер маршрута.

3.1 Лабораторная работа № 1

Алгоритм генерации графа реализован методом `generate`, приведённым в листинге 1. Он формирует связный ориентированный граф по последовательности степеней, полученной с помощью распределения Вейбулла.

Вход: количество вершин графа.

Выход: матрицы смежности ориентированного и неориентированного графов.

Сложность: $O(p^2)$.

Листинг 1: Генерация графа

```
1 void GeneratorGraph::generate() {
2     m_adjacencyMatrix = Matrix(m_vertexCount, m_vertexCount);
3     auto degrees = m_generateDegreeSequence();
4
5     // гарантирует связность
6     for (int i = 0; i < m_vertexCount - 1; ++i) {
7         m_adjacencyMatrix(i, i + 1) = 1;
8     }
9
10    // добор оставшихся рёбер
11    for (int i = 0; i < m_vertexCount; ++i) {
12        int alreadyAdded = (i < m_vertexCount - 1) ? 1 : 0;
13
14        std::vector<int> candidates;
15        for (int j = i + 1; j < m_vertexCount; ++j) {
16            // если ещё не добавлено
17            if (m_adjacencyMatrix(i, j) == 0) {
18                candidates.push_back(j);
19            }
20        }
21        std::shuffle(candidates.begin(), candidates.end(), m_rng);
22
23        int needToAdd = std::max(0, degrees[i] - alreadyAdded);
24        int edgesAdded = 0;
25        for (int j : candidates) {
26            if (edgesAdded >= needToAdd) break;
27            m_adjacencyMatrix(i, j) = 1;
28            ++edgesAdded;
```



```

29     }
30 }
31
32 m_syncUndirectedAdjacencyMatrix();
33 isMatrixInit.adjacency = true;
34 }

```

Алгоритм вычисления эксцентриситетов реализован методом `computeEccentricities`, приведённым в листинге 2. По степеням матрицы смежности он находит длины кратчайших маршрутов и вычисляет характеристики вершин.

Вход: матрица смежности графа.

Выход: эксцентриситеты, диаметр, центральные и диаметральные вершины.

Сложность: $O(p^4)$, поскольку выполняется до $p - 1$ умножений матриц.

Листинг 2: Вычисление эксцентриситетов, центра и диаметра

```

1 void GeneratorGraph::computeEccentricities() {
2     const Matrix& A = m_adjacencyMatrix;
3
4     const int UNREACHABLE = -1;
5     Matrix dist(m_vertexCount, m_vertexCount, UNREACHABLE);
6     for (int i = 0; i < m_vertexCount; ++i) dist(i, i) = 0;
7
8     Matrix Ak = A;
9     for (int k = 1; k < m_vertexCount; ++k) {
10         for (int i = 0; i < m_vertexCount; ++i) {
11             for (int j = 0; j < m_vertexCount; ++j) {
12                 if (i != j && dist(i, j) == UNREACHABLE && Ak(i, j) > 0)
13                     ↪ {
14                         dist(i, j) = k;
15                     }
16             }
17             if (k < m_vertexCount - 1) Ak = Ak * A;
18         }
19
20         m_eccentricities.assign(m_vertexCount, 0);
21         for (int i = 0; i < m_vertexCount; ++i) {
22             int maxDist = 0;
23             for (int j = 0; j < m_vertexCount; ++j) {
24                 if (i != j && dist(i, j) != UNREACHABLE) {
25                     maxDist = std::max(maxDist, static_cast<int>(dist(i,
26                     ↪ j)));
27                 }
28             }
29             m_eccentricities[i] = maxDist;
30         }
31     }
32 }

```

```

31     m_diameter = *std::max_element(m_eccentricities.begin(),
    ↪ m_eccentricities.end());
32     int radius = *std::min_element(m_eccentricities.begin(),
    ↪ m_eccentricities.end());
33
34     m_centerVertices.clear();
35     m_diametralVertices.clear();
36     for (int i = 0; i < m_vertexCount; ++i) {
37         if (m_eccentricities[i] == radius)
    ↪ m_centerVertices.push_back(i);
38         if (m_eccentricities[i] == m_diameter)
    ↪ m_diametralVertices.push_back(i);
39     }
40 }

```

Метод `generateWeightMatrix`, приведённый в листинге 3, присваивает существующим рёбрам веса в выбранном режиме.

Вход: матрица смежности и режим генерации положительных, отрицательных или смешанных весов.

Выход: весовые матрицы ориентированного и неориентированного графов.

Листинг 3: Генерация весовой матрицы

```

1 void GeneratorGraph::generateWeightMatrix(WeightMode mode) {
2     const double INF = std::numeric_limits<double>::infinity();
3
4     m_weightMatrix = Matrix(m_vertexCount, m_vertexCount, INF);
5
6     std::uniform_int_distribution<int> signDist(0, 1);
7
8     for (int i = 0; i < m_vertexCount; ++i) {
9         m_weightMatrix(i, i) = 0;
10
11         for (int j = 0; j < m_vertexCount; ++j) {
12             if (i == j) continue;
13
14             if (m_adjacencyMatrix(i, j) != 0) {
15                 const int WEIGHT_MATRIX_DIST_VALUE = 10;
16                 double w = std::round(m_sampleWeibull(constants::weight )
    ↪ * WEIGHT_MATRIX_DIST_VALUE);
17
18                 switch (mode) {
19                     case WeightMode::Positive:
20                         m_weightMatrix(i, j) = w;
21                         break;
22
23                     case WeightMode::Negative:
24                         m_weightMatrix(i, j) = -w;
25                         break;
26

```

```

27         case WeightMode::Mixed:
28             m_weightMatrix(i, j) = signDist(m_rng) ? w : -w;
29             break;
30     }
31 }
32 }
33 }
34
35 isMatrixInit.weight = true;
36 m_syncUndirectedWeightMatrix();
37 }

```

Метод Шимбелла реализован методом `shimbell`, приведённым в листинге 4. Он последовательно строит весовую матрицу маршрутов заданной длины.

Вход: матрицы смежности и весов, количество рёбер маршрута и требуемый тип веса: минимальный либо максимальный.

Выход: матрица весов маршрутов заданной длины.

Сложность: $O(kp^3)$.

Листинг 4: Реализация метода Шимбелла

```

1 Matrix GeneratorGraph::shimbell(int steps, bool findMin) const {
2     const double NO_EDGE = findMin
3         ? std::numeric_limits<double>::infinity()
4         : -std::numeric_limits<double>::infinity();
5
6     Matrix W(m_vertexCount, m_vertexCount, NO_EDGE);
7
8     if (steps == 0) {
9         for (int i = 0; i < m_vertexCount; ++i) {
10             W(i, i) = 0;
11         }
12         return W;
13     }
14
15     for (int i = 0; i < m_vertexCount; ++i) {
16         for (int j = 0; j < m_vertexCount; ++j) {
17             if (m_adjacencyMatrix(i, j) != 0) {
18                 W(i, j) = m_weightMatrix(i, j);
19             }
20         }
21     }
22
23     Matrix result = W;
24
25     for (int s = 1; s < steps; ++s) {
26         result = findMin ? result.shimbellMin(W) : result.shimbellMax(W);
27     }
28 }

```

```

29     for (int i = 0; i < m_vertexCount; ++i) {
30         result(i, i) = 0;
31     }
32
33     return result;
34 }

```

Алгоритм исследования маршрутов реализован методами `hasRoute` и `countRoutes`, приведёнными в листинге 5. Методы используют степени матрицы смежности для исследования маршрутов между двумя вершинами.

Вход: матрица смежности, начальная и конечная вершины.

Выход: признак существования маршрута и количество найденных маршрутов.

Сложность: $O(p^4)$.

Листинг 5: Проверка наличия и подсчёт маршрутов

```

1  bool GeneratorGraph::hasRoute(int from, int to) const {
2      if (from == to) return true;
3
4      Matrix Ak = m_adjacencyMatrix;
5      for (int k = 1; k < m_vertexCount; ++k) {
6          if (Ak(from, to) > 0) return true;
7          if (k < m_vertexCount - 1) Ak = Ak * m_adjacencyMatrix;
8      }
9      return false;
10 }
11
12 int GeneratorGraph::countRoutes(int from, int to) const {
13     if (from == to) return 1;
14
15     int total = 0;
16     Matrix Ak = m_adjacencyMatrix;
17     for (int k = 1; k < m_vertexCount; ++k) {
18         total += static_cast<int>(Ak(from, to));
19         if (k < m_vertexCount - 1) Ak = Ak * m_adjacencyMatrix;
20     }
21     return total;
22 }

```

3.2 Лабораторная работа № 2

Алгоритм поиска точек сочленения реализован методами `m_dfsArticulation` и `findArticulationPoints`, приведёнными в листинге 6. Они выполняют обход графа в глубину, вычисляют время входа и нижние метки вершин.

Вход: матрица смежности.

Выход: список точек сочленения и количество итераций алгоритма.

Сложность: $O(p^2)$ при хранении графа в виде матрицы смежности.

Листинг 6: Поиск точек сочленения

```
1 void GeneratorGraph::m_dfsArticulation(int v,
2                                     int parent,
3                                     std::vector<int>& tin,
4                                     std::vector<int>& low,
5                                     std::vector<bool>& visited,
6                                     std::vector<bool>& isAP,
7                                     int& timer,
8                                     int& iterations) const {
9     visited[v] = true;
10    tin[v] = low[v] = timer++;
11
12    int children = 0;
13
14    for (int u = 0; u < m_vertexCount; ++u) {
15        iterations++;
16
17        if (u == v) continue;
18
19        if (m_adjacencyMatrix(v, u) == 0 && m_adjacencyMatrix(u, v) == 0)
20            ↪ {
21                continue;
22            }
23
24        if (u == parent) {
25            continue;
26        }
27
28        if (visited[u]) {
29            low[v] = std::min(low[v], tin[u]);
30        } else {
31            ++children;
32
33            m_dfsArticulation(u, v, tin, low, visited, isAP, timer,
34                            ↪ iterations);
35
36            low[v] = std::min(low[v], low[u]);
37
38            if (parent != -1 && low[u] >= tin[v]) {
39                isAP[v] = true;
40            }
41        }
42    }
43
44    if (parent == -1 && children > 1) {
45        isAP[v] = true;
46    }
```

```

47 // упрощенный упрощённый алгоритм Тарьяна (лек. 4 стр. 11 - полный)
48 // не использует стек рёбер
49 // квадратичная сложность из-за использования матрицы смежности, со
   ↳ списками сложность  $O(V + E)$ 
50 ArticulationPointsResult GeneratorGraph::findArticulationPoints() const {
51     std::vector<int> tin(m_vertexCount, -1);
52     std::vector<int> low(m_vertexCount, -1);
53     std::vector<bool> visited(m_vertexCount, false);
54     std::vector<bool> isAP(m_vertexCount, false);
55
56     int timer = 0;
57     ArticulationPointsResult result;
58
59     for (int v = 0; v < m_vertexCount; ++v) {
60         if (!visited[v]) {
61             m_dfsArticulation(v, -1, tin, low, visited, isAP, timer,
   ↳ result.iterations);
62         }
63     }
64
65     for (int v = 0; v < m_vertexCount; ++v) {
66         if (isAP[v]) {
67             result.points.push_back(v);
68         }
69     }
70
71     return result;
72 }

```

Модифицированный алгоритм Дейкстры реализован методом `dijkstraNegative`, приведённым в листинге 7. Он выбирает из очереди вершину с минимальным текущим весом, допускает её повторную обработку и контролирует число уменьшений веса для обнаружения отрицательного цикла.

Вход: матрица смежности, весовая матрица, начальная и конечная вершины.

Выход: вектор расстояний, предшественники вершин, кратчайший путь, его длина, число итераций и признак отрицательного цикла.

Сложность: $O(e^p)$ в худшем случае.

Листинг 7: Поиск кратчайшего пути при наличии отрицательных весов

```

1 ShortestPathResult GeneratorGraph::dijkstraNegative(
2     int s,
3     int t,
4     const Matrix& adjacency,
5     const Matrix& weight
6 ) const {
7     const double INF = std::numeric_limits<double>::infinity();
8

```

```

9     ShortestPathResult result;
10
11     // T[v] - длина кратчайшего пути от s к v
12     // H[v] - вершина, предшествующая v по кратчайшему пути
13
14     std::vector<double> T(m_vertexCount, INF);
15     std::vector<int> H(m_vertexCount, -1);
16
17     std::vector<int> relaxCount(m_vertexCount, 0);
18
19     // { T[v], v }
20     using QueueItem = std::pair<double, int>;
21
22     // очередь с приоритетами
23     // сначала извлекается вершина с минимальным T[v]
24     std::priority_queue<
25         QueueItem,
26         std::vector<QueueItem>,
27         std::greater<QueueItem>
28     > Q;
29
30     int iterations = 0;
31
32     T[s] = 0;
33     H[s] = -1;
34
35     Q.push({0, s});
36
37     while (!Q.empty()) {
38         double currentDistance = Q.top().first;
39         int v = Q.top().second;
40
41         Q.pop();
42
43         // пропускаем устаревшую запись
44         if (currentDistance != T[v]) {
45             continue;
46         }
47
48         for (int u = 0; u < m_vertexCount; ++u) {
49             ++iterations;
50
51             if (adjacency(v, u) == 0) {
52                 continue;
53             }
54
55             // по нер-ву треугольника
56             if (T[v] != INF && T[u] > T[v] + weight(v, u)) {
57                 T[u] = T[v] + weight(v, u);
58                 H[u] = v;
59
60                 Q.push({T[u], u});
61

```

```

62         ++relaxCount[u];
63
64         // выходим, если пошли по циклу
65         if (relaxCount[u] >= m_vertexCount) {
66             result.hasNegativeCycle = true;
67             break;
68         }
69     }
70 }
71
72     if (result.hasNegativeCycle) {
73         break;
74     }
75 }
76
77 result.T = T;
78 result.H = H;
79
80 result.iterations = iterations;
81
82 result.hasPath = T[t] != INF;
83 result.distance = T[t];
84
85 if (result.hasPath && !result.hasNegativeCycle) {
86     result.path = m_restorePath(s, t, H);
87 }
88
89 return result;
90 }

```

3.3 Лабораторная работа № 3

Метод `generateCapacityAndCostMatrices`, приведённый в листинге 8, назначает каждому ребру исходного ориентированного графа пропускную способность и стоимость.

Вход: матрица смежности ориентированного графа.

Выход: матрицы пропускных способностей и стоимостей.

Листинг 8: Генерация пропускных способностей и стоимостей

```

1 void GeneratorGraph::generateCapacityAndCostMatrices() {
2     m_capacityMatrix = Matrix(m_vertexCount, m_vertexCount, 0);
3     m_costMatrix = Matrix(m_vertexCount, m_vertexCount, 0);
4
5     for (int i = 0; i < m_vertexCount; ++i) {
6         for (int j = 0; j < m_vertexCount; ++j) {
7             if (m_adjacencyMatrix(i, j) != 0) {
8                 double capacityRow =
9                     ↪ m_sampleWeibull(constants::capacity);
10                double costRow = m_sampleWeibull(constants::cost);

```



```

11         int capacity = std::max(1,
    ↪ static_cast<int>(std::round(capacityRaw * 10)));
12         int cost = std::max(1,
    ↪ static_cast<int>(std::round(costRaw * 10)));
13
14         m_capacityMatrix(i, j) = capacity;
15         m_costMatrix(i, j) = cost;
16     }
17 }
18 }
19
20 isMatrixInit.capacity = true;
21 isMatrixInit.cost = true;
22 }

```

Алгоритм Форда — Фалкерсона реализован методом `fordFulkerson`, приведённым в листинге 9. Он расставляет метки, находит увеличивающую цепь и изменяет поток вдоль неё, пока сток может получить метку.

Вход: матрица пропускных способностей, исток и сток.

Выход: величина максимального потока, матрица потоков и количество итераций.

Сложность: $O(Ap^3)$, где A — количество найденных увеличивающих цепей.

Листинг 9: Вычисление максимального потока

```

1 MaxFlowResult GeneratorGraph::fordFulkerson(int source, int sink) const {
2     const int INF = std::numeric_limits<int>::max();
3
4     MaxFlowResult result(m_vertexCount);
5
6     if (!isMatrixInit.capacity) {
7         return result;
8     }
9
10    if (source < 0 || source >= m_vertexCount ||
11        sink < 0 || sink >= m_vertexCount ||
12        source == sink) {
13        return result;
14    }
15
16    struct Mark {
17        char sign = '+';
18        int parent = -1;
19        int delta = 0;
20    };
21
22    while (true) {
23        // S[v] - вершина получила пометку
24        // N[v] - вершина уже обработана

```

```

25      // P[v] - пометка вершины: знак, предок, величина изменения
      ↪ потока
26      std::vector<bool> S(m_vertexCount, false);
27      std::vector<bool> N(m_vertexCount, false);
28      std::vector<Mark> P(m_vertexCount);
29
30      S[source] = true;
31      P[source] = {'+', -1, INF};
32
33      bool expanded = true;
34
35      // расставляем пометки, пока множество S расширяется
36      while (expanded && !S[sink]) {
37          expanded = false;
38
39          for (int v = 0; v < m_vertexCount; ++v) {
40              ++result.iterations;
41
42              if (!S[v] || N[v]) {
43                  continue;
44              }
45
46              // прямые дуги
47              for (int u = 0; u < m_vertexCount; ++u) {
48                  ++result.iterations;
49
50                  if (S[u]) {
51                      continue;
52                  }
53
54                  if (m_capacityMatrix(v, u) <= 0) {
55                      continue;
56                  }
57
58                  int available =
59                      static_cast<int>(m_capacityMatrix(v, u) -
                      ↪ result.flowMatrix(v, u));
60
61                  if (available > 0) {
62                      S[u] = true;
63                      P[u] = {
64                          '+',
65                          v,
66                          std::min(P[v].delta, available)
67                      };
68
69                      expanded = true;
70
71                      if (u == sink) {
72                          break;
73                      }
74                  }
75              }

```

```

76
77         if (S[sink]) {
78             break;
79         }
80
81         // обратные дуги
82         for (int u = 0; u < m_vertexCount; ++u) {
83             ++result.iterations;
84
85             if (S[u]) {
86                 continue;
87             }
88
89             if (result.flowMatrix(u, v) > 0) {
90                 int available =
91                     static_cast<int>(result.flowMatrix(u, v));
92
93                 S[u] = true;
94                 P[u] = {
95                     '-',
96                     v,
97                     std::min(P[v].delta, available)
98                 };
99
100                 expanded = true;
101
102                 if (u == sink) {
103                     break;
104                 }
105             }
106         }
107
108         N[v] = true;
109
110         if (S[sink]) {
111             break;
112         }
113     }
114 }
115
116 // если сток не получил пометку, увеличивающих цепей больше нет
117 if (!S[sink]) {
118     break;
119 }
120
121 int delta = P[sink].delta;
122 int current = sink;
123
124 // изменяем поток по найденной увеличивающей цепи
125 while (current != source) {
126     Mark mark = P[current];
127     int previous = mark.parent;
128

```

```

129         if (mark.sign == '+') {
130             result.flowMatrix(previous, current) += delta;
131         } else {
132             result.flowMatrix(current, previous) -= delta;
133         }
134
135         current = previous;
136     }
137
138     result.maxFlow += delta;
139 }
140
141 return result;
142 }
143
144 // поток минимальной стоимости
145 // заданная величина потока передаётся параметром requiredFlow

```

Алгоритм последовательного нахождения потока минимальной стоимости реализован методом `minCostFlow`, приведённым в листинге 10. Он находит кратчайшие пути в сети и изменяет поток до достижения требуемой величины либо отсутствия подходящего пути.

Вход: матрицы пропускных способностей и стоимостей, исток, сток и требуемая величина потока.

Выход: достигнутая величина и стоимость потока, матрицы потоков и стоимостей потока, число итераций и признак успешного завершения.

Сложность: $O(Ae^p)$ в худшем случае, где A — количество увеличений потока. Экспоненциальная сложность обусловлена использованием модифицированного алгоритма Дейкстры при каждом увеличении потока.

Листинг 10: Вычисление потока минимальной стоимости

```

1  MinCostFlowResult GeneratorGraph::minCostFlow(int source,
2                                              int sink,
3                                              int requiredFlow) const {
4      MinCostFlowResult result(m_vertexCount);
5
6      result.requiredFlow = requiredFlow;
7
8      if (!isMatrixInit.capacity || !isMatrixInit.cost) {
9          return result;
10     }
11
12     if (source < 0 || source >= m_vertexCount ||
13         sink < 0 || sink >= m_vertexCount ||
14         source == sink ||
15         requiredFlow <= 0) {
16         return result;
17     }
18

```

```

19 Matrix residualCapacity = m_capacityMatrix;
20 Matrix residualCost(m_vertexCount, m_vertexCount, 0);
21
22 for (int i = 0; i < m_vertexCount; ++i) {
23     for (int j = 0; j < m_vertexCount; ++j) {
24         if (m_capacityMatrix(i, j) > 0) {
25             residualCost(i, j) = m_costMatrix(i, j);
26             residualCost(j, i) = -m_costMatrix(i, j);
27         }
28     }
29 }
30
31 while (result.achievedFlow < requiredFlow) {
32     Matrix residualAdjacency(m_vertexCount, m_vertexCount, 0);
33
34     for (int i = 0; i < m_vertexCount; ++i) {
35         for (int j = 0; j < m_vertexCount; ++j) {
36             if (residualCapacity(i, j) > 0) {
37                 residualAdjacency(i, j) = 1;
38             }
39         }
40     }
41
42     auto pathResult = dijkstraNegative(
43         source,
44         sink,
45         residualAdjacency,
46         residualCost
47     );
48
49     result.iterations += pathResult.iterations;
50
51     if (!pathResult.hasPath || pathResult.hasNegativeCycle) {
52         break;
53     }
54
55     int addFlow = requiredFlow - result.achievedFlow;
56
57     for (size_t i = 1; i < pathResult.path.size(); ++i) {
58         int from = pathResult.path[i - 1];
59         int to = pathResult.path[i];
60
61         addFlow = std::min(
62             addFlow,
63             static_cast<int>(residualCapacity(from, to))
64         );
65     }
66
67     if (addFlow <= 0) {
68         break;
69     }
70
71     for (size_t i = 1; i < pathResult.path.size(); ++i) {

```

```

72         int from = pathResult.path[i - 1];
73         int to = pathResult.path[i];
74
75         residualCapacity(from, to) -= addFlow;
76         residualCapacity(to, from) += addFlow;
77
78         if (m_capacityMatrix(from, to) > 0) {
79             // идём по исходной дуге
80             result.flowMatrix(from, to) += addFlow;
81             result.totalCost += addFlow *
82                 ↪ static_cast<int>(m_costMatrix(from, to));
83         } else {
84             // идём по обратной дуге остаточной сети
85             // значит уменьшаем поток по исходной дуге to -> from
86             result.flowMatrix(to, from) -= addFlow;
87             result.totalCost -= addFlow *
88                 ↪ static_cast<int>(m_costMatrix(to, from));
89         }
90     }
91     result.achievedFlow += addFlow;
92 }
93 result.success = result.achievedFlow == requiredFlow;
94
95 for (int i = 0; i < m_vertexCount; ++i) {
96     for (int j = 0; j < m_vertexCount; ++j) {
97         result.costFlowMatrix(i, j) =
98             m_costMatrix(i, j) * result.flowMatrix(i, j);
99     }
100 }
101
102 return result;
103 }

```

3.4 Лабораторная работа № 4

Алгоритм подсчёта остовных деревьев по теореме Кирхгофа реализован методом `countSpanningTreesKirchhoff`, приведённым в листинге 11. Он строит матрицу Лапласа и вычисляет определитель её минора.

Вход: матрица смежности неориентированного графа.

Выход: матрица Лапласа, её минор и число остовных деревьев.

Сложность: $O(p!)$, поскольку определитель вычисляется рекурсивным разложением по строке.

Листинг 11: Подсчёт остовных деревьев по теореме Кирхгофа

```

1 SpanningTreesResult GeneratorGraph::countSpanningTreesKirchhoff() const {
2     SpanningTreesResult result(m_vertexCount);
3     const Matrix& adjacency = m_undirectedAdjacencyMatrix;

```

```

4
5     for (int i = 0; i < m_vertexCount; ++i) {
6         int degree = 0;
7
8         for (int j = 0; j < m_vertexCount; ++j) {
9             if (i == j) {
10                 continue;
11             }
12
13             if (adjacency(i, j) != 0) {
14                 ++degree;
15                 result.kirchhoffMatrix(i, j) = -1;
16             }
17         }
18
19         result.kirchhoffMatrix(i, i) = degree;
20     }
21
22     if (m_vertexCount <= 1) {
23         result.count = 1;
24         return result;
25     }
26
27     for (int i = 1; i < m_vertexCount; ++i) {
28         for (int j = 1; j < m_vertexCount; ++j) {
29             result.cofactorMatrix(i - 1, j - 1) =
30                 result.kirchhoffMatrix(i, j);
31         }
32     }
33
34     result.cofactorDeterminant = result.cofactorMatrix.determinant();
35     result.count = std::llround(result.cofactorDeterminant);
36
37     return result;
38 }

```

Определитель вычисляется методом `determinant`, приведённым в листинге 12. Метод проверяет, что матрица является квадратной, и вызывает рекурсивное разложение, приведённое в листинге 13. Для сокращения количества слагаемых выбирается строка или столбец с наибольшим числом нулевых элементов.

Вход: квадратная матрица.

Выход: определитель матрицы.

Сложность: $O(p!)$ в худшем случае.

Листинг 12: Метод вычисления определителя

```

1 double Matrix::determinant() const {
2     if (m_rows != m_cols) {
3         throw std::invalid_argument(
4             "determinant is defined only for square matrices"

```

```

5         );
6     }
7
8     return determinantRecursive(m_data);
9 }
10

```

Листинг 13: Рекурсивное разложение определителя

```

1
2 std::vector<std::vector<double>> buildMinor(
3     const std::vector<std::vector<double>>& data,
4     int excludedRow,
5     int excludedCol
6 ) {
7     const int n = static_cast<int>(data.size());
8     std::vector<std::vector<double>> minor;
9     minor.reserve(n - 1);
10
11     for (int i = 0; i < n; ++i) {
12         if (i == excludedRow) {
13             continue;
14         }
15
16         std::vector<double> row;
17         row.reserve(n - 1);
18
19         for (int j = 0; j < n; ++j) {
20             if (j != excludedCol) {
21                 row.push_back(data[i][j]);
22             }
23         }
24
25         minor.push_back(row);
26     }
27
28     return minor;
29 }
30
31 double determinantRecursive(
32     const std::vector<std::vector<double>>& data
33 ) {
34     const int n = static_cast<int>(data.size());
35
36     if (n == 0) {
37         return 1;
38     }
39
40     if (n == 1) {
41         return data[0][0];
42     }
43
44     if (n == 2) {
45         return data[0][0] * data[1][1]
46             - data[0][1] * data[1][0];
47     }
48
49     bool expandByRow = true;
50     int expandIndex = 0;
51     int maxZeros = -1;
52

```



```

53     for (int i = 0; i < n; ++i) {
54         int zeros = 0;
55
56         for (int j = 0; j < n; ++j) {
57             if (isZero(data[i][j])) {
58                 ++zeros;
59             }
60         }
61
62         if (zeros > maxZeros) {
63             maxZeros = zeros;
64             expandByRow = true;
65             expandIndex = i;
66         }
67     }
68
69     for (int j = 0; j < n; ++j) {
70         int zeros = 0;
71
72         for (int i = 0; i < n; ++i) {
73             if (isZero(data[i][j])) {
74                 ++zeros;
75             }
76         }
77
78         if (zeros > maxZeros) {
79             maxZeros = zeros;
80             expandByRow = false;
81             expandIndex = j;
82         }
83     }
84
85     if (maxZeros == n) {
86         return 0;
87     }
88
89     double determinant = 0;
90
91     if (expandByRow) {
92         const int row = expandIndex;
93
94         for (int col = 0; col < n; ++col) {
95             if (isZero(data[row][col])) {
96                 continue;
97             }
98
99             const double sign = ((row + col) % 2 == 0) ? 1 : -1;
100             determinant += sign
101                 * data[row][col]
102                 * determinantRecursive(buildMinor(data, row, col));
103         }
104     } else {
105         const int col = expandIndex;
106
107         for (int row = 0; row < n; ++row) {
108             if (isZero(data[row][col])) {
109                 continue;
110             }
111
112             const double sign = ((row + col) % 2 == 0) ? 1 : -1;
113             determinant += sign
114                 * data[row][col]

```

```

115         * determinantRecursive(buildMinor(data, row, col));
116     }
117 }
118
119     return determinant;
120 }
121 }

```

Алгоритм Краскала реализован методами `m_kruskalAlgorithm` и `kruskalMinimumSpanningTree`, приведёнными в листинге 14. Они сортируют рёбра, объединяют множества вершин и формируют минимальный остов.

Вход: матрицы смежности и весов неориентированного графа.

Выход: рёбра и вес минимального остова, его весовая матрица и признак успешного построения.

Сложность: $O(p^2 + q \log q)$ с учётом формирования списка рёбер из матрицы смежности.

Листинг 14: Построение минимального остова алгоритмом Краскала

```

1  KruskalResult GeneratorGraph::m_kruskalAlgorithm() const {
2      KruskalResult result(m_vertexCount);
3      std::vector<WeightedEdge> edges;
4
5      for (int i = 0; i < m_vertexCount; ++i) {
6          result.treeMatrix(i, i) = 0;
7      }
8
9      // берем все ребра графа
10     for (int i = 0; i < m_vertexCount; ++i) {
11         for (int j = i + 1; j < m_vertexCount; ++j) {
12             if (m_undirectedAdjacencyMatrix(i, j) != 0 &&
13                 !std::isinf(m_undirectedWeightMatrix(i, j))) {
14                 edges.push_back({
15                     i,
16                     j,
17                     m_undirectedWeightMatrix(i, j)
18                 });
19             }
20         }
21     }
22
23     // сортируем рёбра по возрастанию веса
24     std::sort(
25         edges.begin(),
26         edges.end(),
27         [](const WeightedEdge& lhs, const WeightedEdge& rhs) {
28             if (lhs.weight != rhs.weight) {
29                 return lhs.weight < rhs.weight;
30             }
31         }

```

```

32         if (lhs.from != rhs.from) {
33             return lhs.from < rhs.from;
34         }
35
36         return lhs.to < rhs.to;
37     }
38 );
39
40     std::vector<int> parent(m_vertexCount);
41     std::vector<int> rank(m_vertexCount, 0);
42
43     for (int i = 0; i < m_vertexCount; ++i) {
44         parent[i] = i;
45     }
46
47     std::function<int(int)> findSet = [&](int vertex) {
48         if (parent[vertex] != vertex) {
49             parent[vertex] = findSet(parent[vertex]);
50         }
51
52         return parent[vertex];
53     };
54
55     auto unionSets = [&](int first, int second) {
56         first = findSet(first);
57         second = findSet(second);
58
59         if (first == second) {
60             return false;
61         }
62
63         if (rank[first] < rank[second]) {
64             std::swap(first, second);
65         }
66
67         parent[second] = first;
68
69         if (rank[first] == rank[second]) {
70             ++rank[first];
71         }
72
73         // если ребро соединяет вершины из разных множеств, его можно
74         → взять
75         return true;
76     };
77
78     // идем по ребрам от самого дешевого к самому дорогому
79     // изначально каждая вершина в своем множестве [сверху как раз
80     → сделаны ф. поиска и пересечения «множеств»]
81     for (const auto& edge : edges) {
82         if (unionSets(edge.from, edge.to)) {
83             // добавляем ребро в остов, если оно не образует цикл
84             result.edges.push_back(edge);

```

```

83         result.treeMatrix(edge.from, edge.to) = edge.weight;
84         result.treeMatrix(edge.to, edge.from) = edge.weight;
85         result.totalWeight += edge.weight;
86
87         // останавливаемся, когда выбрали n - 1 ребро
88         if (static_cast<int>(result.edges.size()) == m_vertexCount -
89             ↪ 1) {
89             break;
90         }
91     }
92 }
93
94 result.success =
95     static_cast<int>(result.edges.size()) == m_vertexCount - 1;
96
97 return result;
98 }
99
100 KruskalMinimumSpanningTreeResult
101 ↪ GeneratorGraph::kruskalMinimumSpanningTree() const {
102     KruskalMinimumSpanningTreeResult
103     ↪ result(std::move(m_kruskalAlgorithm()), m_vertexCount);
104
105     for (int i = 0; i < m_vertexCount; ++i) {
106         result.decodedTreeMatrix(i, i) = 0;
107     }
108
109     if (!result.kruskal.success) {
110         return result;
111     }
112
113     result.pruferCode = m_encodePruferCode(result.kruskal.edges);
114     result.decodedTreeMatrix = m_decodePruferCode(result.pruferCode);
115     result.pruferRoundTripSuccess =
116         matricesEqual(result.kruskal.treeMatrix,
117             ↪ result.decodedTreeMatrix);
118
119     return result;
120 }

```

Алгоритмы кодирования и декодирования кода Прюфера реализованы методами `m_encodePruferCode` и `m_decodePruferCode`, приведёнными в листинге 15. Они кодируют минимальный остов и восстанавливают его с сохранением весов рёбер.

Вход: минимальный остов при кодировании или код Прюфера при декодировании.

Выход: код Прюфера либо восстановленная весовая матрица остова.

Сложность: $O(p^2)$ для кодирования и $O(p^2 \log p)$ для декодирования.

Листинг 15: Кодирование и декодирование остова

```

1  std::vector<PruferCodeItem> GeneratorGraph::m_encodePruferCode(
2      const std::vector<WeightedEdge>& treeEdges
3  ) const {
4      // кодируем полученный остов кодом Прюфера с сохранением весов рёбер
5      std::vector<PruferCodeItem> code;
6      std::vector<std::set<std::pair<int, double>>>
7          ↪ adjacency(m_vertexCount);
8
9      for (const auto& edge : treeEdges) {
10         adjacency[edge.from].insert({edge.to, edge.weight});
11         adjacency[edge.to].insert({edge.from, edge.weight});
12     }
13
14     for (int step = 0; step < m_vertexCount - 1; ++step) {
15         int leaf = -1;
16
17         // на каждом шаге берём лист с минимальным номером
18         for (int vertex = 0; vertex < m_vertexCount; ++vertex) {
19             if (adjacency[vertex].size() == 1) {
20                 leaf = vertex;
21                 break;
22             }
23         }
24
25         if (leaf == -1) {
26             break;
27         }
28
29         const auto [neighbor, weight] = *adjacency[leaf].begin();
30         // в код записываем соседа листа, а рядом сохраняем вес
31         ↪ удаляемого ребра
32         code.push_back({neighbor, weight});
33
34         // удаляем лист из текущего дерева
35         adjacency[neighbor].erase({leaf, weight});
36         adjacency[leaf].clear();
37     }
38
39     return code;
40 }
41
42 Matrix GeneratorGraph::m_decodePruferCode(
43     const std::vector<PruferCodeItem>& code
44 ) const {
45     // декодируем код Прюфера обратно в матрицу весов остова
46     Matrix decodedTreeMatrix(
47         m_vertexCount,
48         m_vertexCount,
49         std::numeric_limits<double>::infinity()
50     );
51     std::vector<int> unusedVertices(m_vertexCount);
52
53     for (int i = 0; i < m_vertexCount; ++i) {

```

```

52     decodedTreeMatrix(i, i) = 0;
53     unusedVertices[i] = i;
54 }
55
56     const int steps = std::min(
57         m_vertexCount - 1,
58         static_cast<int>(code.size())
59     );
60
61     for (int i = 0; i < steps; ++i) {
62         std::set<int> remainingCode;
63
64         for (int j = i; j < static_cast<int>(code.size()); ++j) {
65             remainingCode.insert(code[j].vertex);
66         }
67
68         // выбираем минимальную вершину, которой нет в оставшемся коде
69         auto leafIt = std::find_if(
70             unusedVertices.begin(),
71             unusedVertices.end(),
72             [&](int vertex) {
73                 return remainingCode.find(vertex) == remainingCode.end();
74             }
75         );
76
77         if (leafIt == unusedVertices.end()) {
78             break;
79         }
80
81         const int leaf = *leafIt;
82         const int neighbor = code[i].vertex;
83         const double weight = code[i].weight;
84
85         // восстанавливаем ребро с тем же весом, который был сохранён при
86         → кодировании
87         decodedTreeMatrix(leaf, neighbor) = weight;
88         decodedTreeMatrix(neighbor, leaf) = weight;
89
90         unusedVertices.erase(leafIt);
91     }
92     return decodedTreeMatrix;
93 }

```

Алгоритм Эдмондса реализован методами `edmondsBlossom` и `InOriginalGraph`, `edmondsBlossomInMinimumSpanningTree` и `m_edmondsBlossom`, приведёнными в листинге 16. Они выбирают исследуемый граф и находят максимальное независимое множество рёбер, сжимая обнаруженные нечётные циклы.

Вход: матрицы смежности и весов исходного графа либо минимального остова.

Выход: список и матрица рёбер максимального паросочетания.

Сложность: $O(p^3)$.

Листинг 16: Поиск максимального независимого множества рёбер

```
1 EdmondsBlossomResult
2 GeneratorGraph::edmondsBlossomInOriginalGraph() const {
3     const Matrix& weights = isMatrixInit.weight
4         ? m_undirectedWeightMatrix
5         : m_undirectedAdjacencyMatrix;
6
7     return m_edmondsBlossom(
8         m_undirectedAdjacencyMatrix,
9         weights,
10        isMatrixInit.weight
11    );
12 }
13
14 EdmondsBlossomResult
15 GeneratorGraph::edmondsBlossomInMinimumSpanningTree() const {
16     EdmondsBlossomResult result(m_vertexCount);
17     auto kruskalMinimumSpanningTreeResult = kruskalMinimumSpanningTree();
18
19     if (!kruskalMinimumSpanningTreeResult.kruskal.success) {
20         result.success = false;
21         return result;
22     }
23
24     Matrix adjacency(m_vertexCount, m_vertexCount, 0);
25
26     for (const auto& edge :
27         ↪ kruskalMinimumSpanningTreeResult.kruskal.edges) {
28         adjacency(edge.from, edge.to) = 1;
29         adjacency(edge.to, edge.from) = 1;
30     }
31
32     return m_edmondsBlossom(
33         adjacency,
34         kruskalMinimumSpanningTreeResult.kruskal.treeMatrix,
35         true
36     );
37 }
38 EdmondsBlossomResult GeneratorGraph::m_edmondsBlossom(
39     const Matrix& adjacency,
40     const Matrix& weights,
41     bool hasWeights
42 ) const {
43     EdmondsBlossomResult result(m_vertexCount);
44     result.hasWeights = hasWeights;
45
46     if (hasWeights) {
```

```

47     result.edgeMatrix = Matrix(
48         m_vertexCount,
49         m_vertexCount,
50         std::numeric_limits<double>::infinity()
51     );
52
53     for (int i = 0; i < m_vertexCount; ++i) {
54         result.edgeMatrix(i, i) = 0;
55     }
56 }
57
58 const auto graph = m_matrixToAdjacencyList<int>(
59     adjacency,
60     [](int, int to) {
61         return to;
62     }
63 );
64
65 // хотим выбрать как можно больше рёбер, чтобы они не касались друг
66 // => ищем увел. путь, путь от одной вершины до другой, где рёбра
67 //    идут по очереди
68 // BEGIN
69 // а нечётные циклы временно сжимаем.
70 // match[v]: с какой вершиной соединена вершина v
71 std::vector<int> match(m_vertexCount, -1);
72 // parent[v]: предыдущая вершина в дереве поиска увеличивающегося
73 //    пути
74 std::vector<int> parent(m_vertexCount);
75 std::vector<int> base(m_vertexCount);
76 // помечаем, что вершина уже добавлена в очередь
77 std::vector<bool> used(m_vertexCount);
78 // помечаем цикл в графе
79 std::vector<bool> blossom(m_vertexCount);
80 std::queue<int> queue;
81
82 // ищем общую базу нечётного цикла, чтобы сжать к одной вершине
83 auto findLca = [&](int first, int second) {
84     std::vector<bool> usedPath(m_vertexCount, false);
85
86     while (true) {
87         first = base[first];
88         usedPath[first] = true;
89
90         if (match[first] == -1) {
91             break;
92         }
93
94         first = parent[match[first]];
95     }
96
97     while (true) {

```



```

97         second = base[second];
98
99         if (usedPath[second]) {
100             return second;
101         }
102
103         second = parent[match[second]];
104     }
105 };
106
107 auto markPath = [&](int vertex,
108                   int lca,
109                   int child,
110                   auto&& markPathRef) -> void {
111     // помечаем путь от вершины до общей базы
112     while (base[vertex] != lca) {
113         blossom[base[vertex]] = true;
114         blossom[base[match[vertex]]] = true;
115         parent[vertex] = child;
116         child = match[vertex];
117         vertex = parent[match[vertex]];
118     }
119 };
120
121 auto findPath = [&](int root) {
122     // T <- ∅: очищаем BFS-дерево поиска увеличивающего пути
123     std::fill(used.begin(), used.end(), false);
124     std::fill(parent.begin(), parent.end(), -1);
125
126     for (int i = 0; i < m_vertexCount; ++i) {
127         base[i] = i;
128     }
129
130     while (!queue.empty()) {
131         queue.pop();
132     }
133
134     // pick r in F; queue.push(r); T.add(r)
135     queue.push(root);
136     used[root] = true;
137
138     // WHILE queue != ∅
139     while (!queue.empty()) {
140         // v <- queue.pop()
141         int vertex = queue.front();
142         queue.pop();
143
144         // FOR ALL neighbors w of v DO
145         for (int to : graph[vertex]) {
146             if (base[vertex] == base[to] ||
147                 match[vertex] == to) {
148                 continue;
149             }

```

```

150
151     if (to == root ||
152         (match[to] != -1 && parent[match[to]] != -1)) {
153         // ELSE IF w in T AND odd-length cycle detected THEN
154         int lca = findLca(vertex, to);
155         std::fill(blossom.begin(), blossom.end(), false);
156
157         // contract cycle: найден нечётный цикл, сжимаем
158         // → blossom к общей базе
159         markPath(vertex, lca, to, markPath);
160         markPath(to, lca, vertex, markPath);
161
162         for (int i = 0; i < m_vertexCount; ++i) {
163             if (blossom[base[i]]) {
164                 base[i] = lca;
165
166                 if (!used[i]) {
167                     used[i] = true;
168                     queue.push(i);
169                 }
170             }
171         }
172     } else if (parent[to] == -1) {
173         // IF w not in T AND w matched THEN T.add(w),
174         // → T.add(mate(w))
175         parent[to] = vertex;
176
177         if (match[to] == -1) {
178             // ELSE IF w in F THEN свободная вершина
179             // → завершает увеличивающий путь
180             return to;
181         }
182
183         // queue.push(mate(w))
184         int next = match[to];
185         used[next] = true;
186         queue.push(next);
187     }
188 }
189
190 return -1;
191 };
192
193 auto augment = [&](int vertex) {
194     // expand all contracted nodes; reconstruct augmenting path;
195     // invert augmenting path
196     while (vertex != -1) {
197         int previous = parent[vertex];
198         int next = (previous == -1) ? -1 : match[previous];
199         match[vertex] = previous;

```

```

200         if (previous != -1) {
201             match[previous] = vertex;
202         }
203
204         vertex = next;
205     }
206 };
207
208 // WHILE F != ∅ DO: пробуем стартовать из каждой свободной вершины
209 for (int vertex = 0; vertex < m_vertexCount; ++vertex) {
210     if (match[vertex] == -1) {
211         int endpoint = findPath(vertex);
212
213         if (endpoint != -1) {
214             augment(endpoint);
215         }
216     }
217 }
218 // END
219
220 for (int from = 0; from < m_vertexCount; ++from) {
221     int to = match[from];
222
223     if (to == -1 || from > to) {
224         continue;
225     }
226
227     double value = hasWeights ? weights(from, to) : 1;
228
229     result.edges.push_back({from, to, value});
230     result.edgeMatrix(from, to) = value;
231     result.edgeMatrix(to, from) = value;
232 }
233
234 return result;
235 }

```

3.5 Лабораторная работа № 5

Алгоритм приведения графа к эйлерову виду реализован методом `checkIfEulerianGraph`, приведённым в листинге 17. Он проверяет чётность степеней и попарно добавляет либо удаляет рёбра между нечётными вершинами. Перед удалением проверяется наличие обходного пути, поэтому преобразование прекращается, если оно требует удаления моста.

Вход: матрица смежности неориентированного графа.

Выход: изменённая матрица смежности, списки добавленных и удалённых рёбер, признаки исходного состояния и завершения преобразования.

Сложность: в худшем случае $O(p^6)$ с учётом перебора пар нечётных вершин и повторных поисков обходного пути.

Листинг 17: Проверка и преобразование графа к эйлерову виду

```

1  CheckIfEulerianGraphResult GeneratorGraph::checkIfEulerianGraph(
2      bool allowDisconnectedResult
3  ) const {
4      bool isEulerian = true;
5      bool transformationCompleted = true;
6      bool resultIsConnected = true;
7      Matrix adjacency = getUndirectedAdjacencyMatrix();
8      std::vector<Edge> addedEdges;
9      std::vector<Edge> removedEdges;
10     auto getVertexDegree = [&](size_t i) {
11         int degree = 0;
12         for (size_t j = 0; j < adjacency.cols(); ++j) {
13             if (i == j) continue;
14
15             if (adjacency(i, j) != 0) ++degree;
16         }
17
18         return degree;
19     };
20
21     std::vector<int> verticesWithOddDegrees;
22
23     while (true) {
24         verticesWithOddDegrees.clear();
25
26         for (size_t i = 0; i < adjacency.rows(); ++i) {
27             if (getVertexDegree(i) % 2 != 0) {
28                 isEulerian = false;
29                 verticesWithOddDegrees.push_back(i);
30             }
31         }
32
33         if (verticesWithOddDegrees.empty()) {
34             break;
35         }
36
37         bool graphChanged = false;
38
39         for (size_t i = 0; i < verticesWithOddDegrees.size() &&
40             ↪ !graphChanged; ++i) {
41             for (size_t j = i + 1; j < verticesWithOddDegrees.size();
42                 ↪ ++j) {
43                 const auto u = verticesWithOddDegrees[i];
44                 const auto v = verticesWithOddDegrees[j];
45
46                 if (adjacency(u, v) == 0) {
47                     adjacency(u, v) = 1;
48                     adjacency(v, u) = 1;
49                     addedEdges.emplace_back(u, v);
50                     graphChanged = true;
51                     break;

```

```

50         }
51
52         Matrix temp = adjacency;
53         temp(u, v) = 0;
54         temp(v, u) = 0;
55
56         const auto resultPath = dijkstraNegative(u, v, temp);
57
58         if (resultPath.hasPath || allowDisconnectedResult) {
59             adjacency(u, v) = 0;
60             adjacency(v, u) = 0;
61             removedEdges.emplace_back(u, v);
62
63             if (!resultPath.hasPath) {
64                 resultIsConnected = false;
65             }
66
67             graphChanged = true;
68             break;
69         }
70     }
71 }
72
73 if (!graphChanged) {
74     transformationCompleted = false;
75     break;
76 }
77 }
78
79 FleuryResult eulerianCycle;
80
81 if (transformationCompleted && resultIsConnected) {
82     eulerianCycle = m_flueryAlgorithm(adjacency);
83 }
84
85 return CheckIfEulerianGraphResult(
86     isEulerian,
87     transformationCompleted,
88     std::move(addedEdges),
89     std::move(removedEdges),
90     std::move(eulerianCycle),
91     std::move(adjacency)
92 );
93 }

```

Алгоритм Флёрри реализован методом `m_flueryAlgorithm`, приведённым в листинге 18. Он последовательно выбирает соседнюю вершину, удаляет пройденное ребро и добавляет вершину в результат после исчерпания её соседей.

Вход: матрица смежности эйлерова графа.

Выход: последовательность вершин эйлерова цикла.

Сложность: $O(pq)$ в худшем случае.

Листинг 18: Построение эйлерова цикла

```
1 FleuryResult GeneratorGraph::m_flueryAlgorithm(const Matrix& adjacency)
  ↪ const {
2     auto graph = m_matrixToAdjacencyList<int>(
3         adjacency,
4         [](int, int to) {
5             return to;
6         }
7     );
8
9     FleuryResult path;
10    std::stack<int> stack;
11    stack.push(0);
12
13    while (!stack.empty()) {
14        const int node = stack.top();
15
16        if (graph[node].empty()) {
17            path.emplace_back(node);
18            stack.pop();
19
20        } else {
21            const int next = graph[node].front();
22            graph[node].erase(graph[node].begin());
23
24            const auto reverseEdge = std::find(
25                graph[next].begin(),
26                graph[next].end(),
27                node
28            );
29
30            if (reverseEdge == graph[next].end()) {
31                throw std::logic_error(
32                    "Обратное ребро отсутствует в списке смежности"
33                );
34            }
35
36            graph[next].erase(reverseEdge);
37            stack.push(next);
38        }
39    }
40
41    std::reverse(path.begin(), path.end());
42    return path;
43 }
```

Алгоритм построения фундаментальной системы разрезов реализован методом `buildFundamentalCutSystem`, приведённым в листинге 19. Он строит минимальный остов и для каждого его ребра получает фунда-

ментальный разрез.

Вход: матрицы смежности и весов неориентированного графа.

Выход: минимальный остов и фундаментальная система разрезов.

Сложность: $O(p^3 + q \log q)$.

Листинг 19: Построение фундаментальной системы разрезов

```
1 FundamentalCutsResult GeneratorGraph::buildFundamentalCutSystem() const {
2     FundamentalCutsResult result(m_vertexCount);
3
4     if (!isMatrixInit.adjacency || !isMatrixInit.weight) {
5         return result;
6     }
7
8     result.spanningTree = m_kruskalAlgorithm();
9
10    if (!result.spanningTree.success) {
11        return result;
12    }
13
14    AdjacencyList tree(m_vertexCount);
15
16    for (const auto& edge : result.spanningTree.edges) {
17        tree[edge.from].push_back(edge.to);
18        tree[edge.to].push_back(edge.from);
19    }
20
21    for (const auto& removedEdge : result.spanningTree.edges) {
22        std::vector<bool> firstComponent(m_vertexCount, false);
23        std::vector<int> stack = {removedEdge.from};
24        firstComponent[removedEdge.from] = true;
25
26        // Удаление ребра остова делит дерево на две компоненты.
27        while (!stack.empty()) {
28            const int vertex = stack.back();
29            stack.pop_back();
30
31            for (int to : tree[vertex]) {
32                const bool isRemovedEdge =
33                    (vertex == removedEdge.from && to == removedEdge.to)
34                    ||
35                    (vertex == removedEdge.to && to == removedEdge.from);
36
37                if (!isRemovedEdge && !firstComponent[to]) {
38                    firstComponent[to] = true;
39                    stack.push_back(to);
40                }
41            }
42        }
43
44        GraphCut cut;
```

```

45     // В разрез входят все рёбра исходного графа между компонентами.
46     for (int from = 0; from < m_vertexCount; ++from) {
47         for (int to = from + 1; to < m_vertexCount; ++to) {
48             if (m_undirectedAdjacencyMatrix(from, to) != 0 &&
49                 firstComponent[from] != firstComponent[to]) {
50                 cut.emplace(from, to);
51             }
52         }
53     }
54
55     result.fundamentalCuts.push_back({
56         {removedEdge.from, removedEdge.to},
57         std::move(cut)
58     });
59 }
60
61 result.success =
62     static_cast<int>(result.fundamentalCuts.size()) == m_vertexCount
63     ↪ - 1;
64 return result;
65 }

```

Алгоритм получения остальных разрезов реализован методами `symmetricDifferenceOfFundamentalCuts` и `m_symmetricDifference`, приведёнными в листинге 20. Они последовательно вычисляют симметрическую разность выбранных фундаментальных разрезов.

Вход: фундаментальная система разрезов и выбранные разрезы.

Выход: множество рёбер результирующего разреза.

Сложность: $O(rq \log q)$, где r — количество выбранных фундаментальных разрезов.

Листинг 20: Получение разреза симметрической разностью

```

1  GraphCut GeneratorGraph::symmetricDifferenceOfFundamentalCuts(
2      const FundamentalCutsResult& system,
3      const std::vector<int>& selectedCutIndices
4  ) const {
5      GraphCut result;
6
7      for (int index : selectedCutIndices) {
8          if (index < 0 || index >=
9              ↪ static_cast<int>(system.fundamentalCuts.size())) {
10             throw std::out_of_range("Номер фундаментального разреза вне
11                 ↪ диапазона");
12         }
13
14         result = m_symmetricDifference(
15             result,
16             system.fundamentalCuts[index].cut
17         );

```



```

16     }
17
18     return result;
19 }
20
21 // этот метод показывает, что используем метод из лекции для вычисления
↪ симм. разности
22 // можно найти на слайде 18 «Подпространства циклов и коциклов»
23 GraphCut GeneratorGraph::m_symmetricDifference(
24     const GraphCut& first,
25     const GraphCut& second
26 ) const {
27     GraphCut result;
28     GraphCut edgeCoordinates = first;
29     edgeCoordinates.insert(second.begin(), second.end());
30
31     // разрез рассматриваем как вектор векторного пространства,
↪ натянутого на множество рёбер
32     // коэффициент равен 1, если ребро входит в разрез, иначе равен 0
33     for (const auto& edge : edgeCoordinates) {
34         const int firstCoefficient = first.count(edge) == 0 ? 0 : 1;
35         const int secondCoefficient = second.count(edge) == 0 ? 0 : 1;
36         const int resultCoefficient =
37             (firstCoefficient + secondCoefficient) % 2;
38
39         if (resultCoefficient == 1) {
40             result.insert(edge);
41         }
42     }
43
44     return result;
45 }

```

4 Результаты работы

Для сравнения было выполнено 100 запусков на случайных графах из 10 вершин. В каждом запуске оба алгоритма работали на одном графе. Количество итераций в отдельных запусках показано на рис. 17.

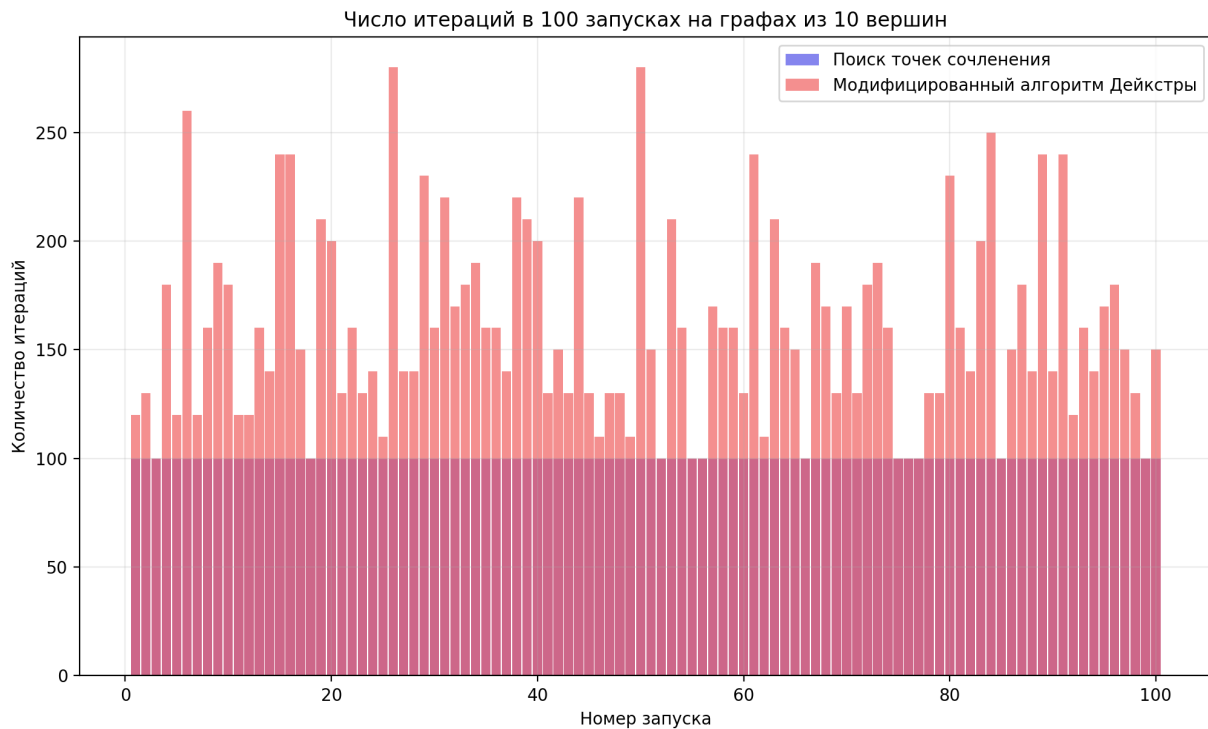


Рис. 17: Количество итераций в каждом запуске

Распределение полученных значений приведено на рис. 18.

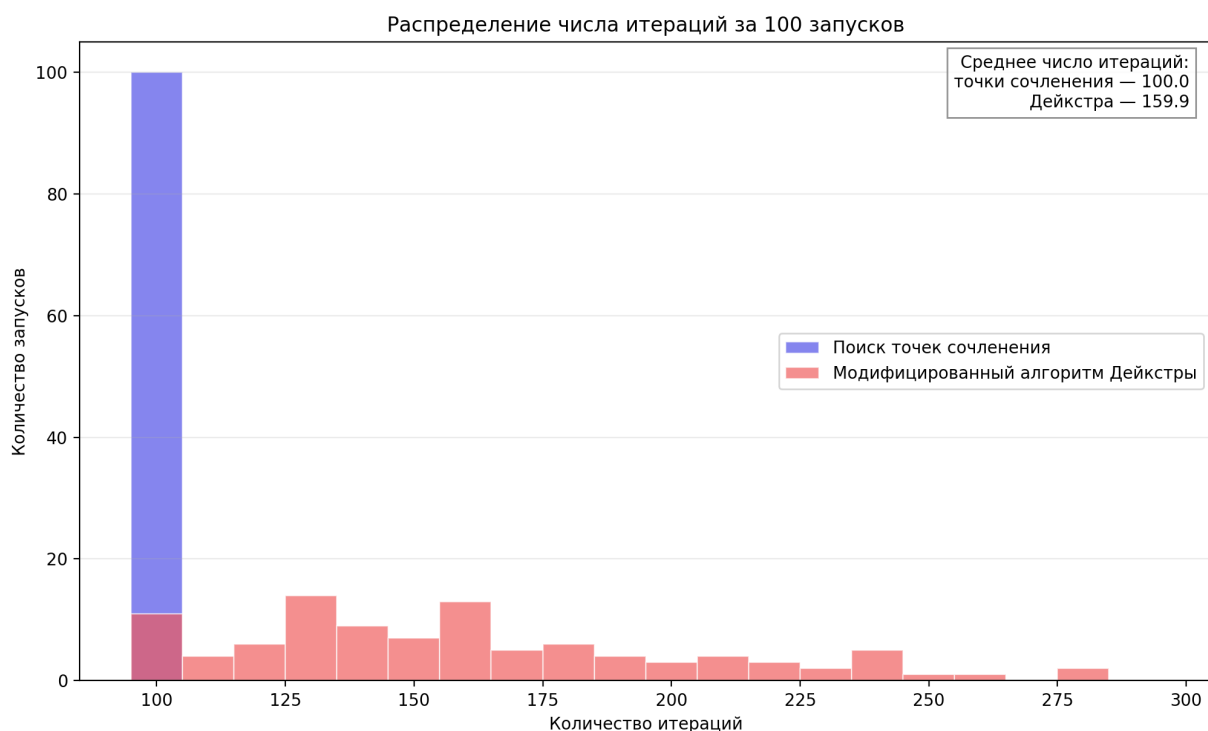


Рис. 18: Распределение количества итераций за 100 запусков

Поиск точек сочленения во всех случаях выполнил 100 итераций. В 89 запусках поиск точек сочленения потребовал меньше итераций, а в 11 запусках значения совпали. Следовательно, на проведённой серии запусков поиск точек сочленения оказался эффективнее по количеству итераций.